BEGINNER → ADVANCED · JOB-READY

Java Backend Revision Guide

Core Java · OOP · Collections · Concurrency · JVM · SQL
Spring Boot · JPA · Security · Testing · Microservices
Kafka · Redis · Docker · DSA · System Design

45	960	150	38
topic areas	checklist items	interview questions	code snippets

Every topic includes

- Key revision notes: the concepts interviewers actually probe
- Code snippets for the patterns you must be able to write
- Interview questions to test yourself out loud
- A tick-box checklist of every sub-topic

Name: _____ Revision started: ____ / ____ / _____

How to use this guide

This guide turns the complete Java syllabus into a revision workbook. Each of the 45 topic areas follows the same layout so you can revise quickly and track progress.

1. **Read the key notes.** Each section opens with the concepts that matter most in real projects and interviews, including the common traps.
2. **Reproduce the code.** Close the guide and type the snippet from memory in your IDE. If you can't, revisit the notes.
3. **Answer the interview check.** Say each answer out loud in under two minutes, as if to an interviewer. Mark weak ones for the next round.
4. **Tick the checklist.** Tick a sub-topic only when you can explain it without notes and give an example.
5. **Use spaced revision.** Revise each topic three times: within a day, after a week, and after a month. Log the dates in the tracker below.

What to prioritise for a Java backend job

You don't need every topic before applying. Work in this order; the last column points to the sections in this guide.

Priority	Focus area	Sections
1	Core Java, OOP, Strings, exceptions	1-10
2	Collections, generics, Streams, lambdas	11-15
3	Multithreading and JVM fundamentals	18-19
4	SQL, JDBC, transactions	21-22
5	Spring Boot, REST APIs, Spring MVC	25-29
6	JPA/Hibernate, SQL optimization	22, 30
7	Spring Security, JWT, OAuth basics	31
8	JUnit, Mockito, integration testing	33
9	Git, Maven, Docker, CI/CD	23-24, 38
10	Microservices, Kafka, Redis, observability	34-37
11	DSA and system design	39-41

Final benchmark: can you independently design, implement, test, debug, deploy and explain a production-quality Spring Boot application? If yes, you are job-ready.

Revision tracker

Write the date of each pass and circle your confidence from 1 (shaky) to 5 (could teach it).

#	Topic area	Pass 1	Pass 2	Pass 3	Confidence
01	Java Fundamentals				1 2 3 4 5
02	Variables and Data Types				1 2 3 4 5
03	Operators				1 2 3 4 5
04	Control Flow Statements				1 2 3 4 5
05	Arrays				1 2 3 4 5
06	Strings				1 2 3 4 5
07	Object-Oriented Programming (OOP)				1 2 3 4 5
08	Access Modifiers and Packages				1 2 3 4 5
09	Exception Handling				1 2 3 4 5
10	Object Class and Core Methods				1 2 3 4 5
11	Java Collections Framework				1 2 3 4 5
12	Generics				1 2 3 4 5
13	Functional Programming				1 2 3 4 5
14	Stream API				1 2 3 4 5
15	Optional API				1 2 3 4 5
16	Date and Time API				1 2 3 4 5
17	File Handling and I/O				1 2 3 4 5
18	Multithreading and Concurrency				1 2 3 4 5
19	JVM Internals				1 2 3 4 5
20	Reflection and Annotations				1 2 3 4 5
21	JDBC				1 2 3 4 5
22	SQL and Database Fundamentals				1 2 3 4 5
23	Build Tools and Dependency Management				1 2 3 4 5
24	Git and Development Tools				1 2 3 4 5
25	Web Fundamentals				1 2 3 4 5
26	Servlets and Jakarta EE				1 2 3 4 5
27	Spring Framework				1 2 3 4 5
28	Spring Boot				1 2 3 4 5
29	Spring MVC and REST APIs				1 2 3 4 5
30	Spring Data JPA and Hibernate				1 2 3 4 5
31	Spring Security				1 2 3 4 5
32	Spring Transactions				1 2 3 4 5
33	Testing Java Applications				1 2 3 4 5
34	Microservices				1 2 3 4 5
35	Caching				1 2 3 4 5
36	Messaging and Event Streaming				1 2 3 4 5
37	Observability				1 2 3 4 5
38	Docker, CI/CD and Cloud				1 2 3 4 5
39	System Design				1 2 3 4 5
40	Design Patterns				1 2 3 4 5
41	Data Structures and Algorithms (DSA)				1 2 3 4 5
42	Advanced Java Internals				1 2 3 4 5
43	Additional Specialized Topics				1 2 3 4 5
44	Java Projects for Practical Learning				1 2 3 4 5
45	Priorities for a Java backend job				1 2 3 4 5

Contents

PART A: Core Java 6

1. Java Fundamentals . . . 6
2. Variables and Data Types . . . 7
3. Operators . . . 8
4. Control Flow Statements . . . 9
5. Arrays . . . 9
6. Strings . . . 10
7. Object-Oriented Programming (OOP) . . . 11
8. Access Modifiers and Packages . . . 13
9. Exception Handling . . . 14
10. Object Class and Core Methods . . . 15
11. Java Collections Framework . . . 16
12. Generics . . . 17
13. Functional Programming . . . 18
14. Stream API . . . 19
15. Optional API . . . 20
16. Date and Time API . . . 21
17. File Handling and I/O . . . 22
18. Multithreading and Concurrency . . . 23
19. JVM Internals . . . 24
20. Reflection and Annotations . . . 26
21. JDBC . . . 27
22. SQL and Database Fundamentals . . . 28
23. Build Tools and Dependency Management . . . 29
24. Git and Development Tools . . . 30

PART B: Java Backend Development 32

25. Web Fundamentals . . . 32
26. Servlets and Jakarta EE . . . 33
27. Spring Framework . . . 34
28. Spring Boot . . . 35
29. Spring MVC and REST APIs . . . 36
30. Spring Data JPA and Hibernate . . . 38
31. Spring Security . . . 39
32. Spring Transactions . . . 41
33. Testing Java Applications . . . 42
34. Microservices . . . 43
35. Caching . . . 44
36. Messaging and Event Streaming . . . 45

37. Observability 46
38. Docker, CI/CD and Cloud 47
39. System Design 49

PART C: Advanced Java Topics **51**

40. Design Patterns 51
41. Data Structures and Algorithms (DSA) 52
42. Advanced Java Internals 54
43. Additional Specialized Topics 55

PART D: Projects and Priorities **57**

44. Java Projects for Practical Learning 57
45. What to Prioritise for a Java Backend Job 58

PART A

Core Java

Language fundamentals, OOP, the standard library, concurrency, the JVM, JDBC, SQL and tooling (topics 1-24).

01 Java Fundamentals

KEY REVISION NOTES

- **JDK vs JRE vs JVM**: the JVM executes bytecode and is platform-specific. The JRE is the JVM plus the core class libraries. The JDK is the JRE plus development tools (`javac`, `jar`, `javadoc`, `jshell`, `jcmd`). Since Java 11 the JRE is no longer shipped separately.
- **Write once, run anywhere**: `.java` source is compiled by `javac` into platform-independent bytecode (`.class`). Each OS has its own JVM that loads, verifies and runs that bytecode (interpreter + JIT compiler).
- **main()**: `public static void main(String[] args)` is public so the JVM can call it, static so no object is needed, and `args` holds command-line arguments. Java 25 finalised compact source files and instance main methods, so `void main()` also works for simple programs.
- **Editions**: Java SE is the standard platform. Jakarta EE (formerly Java EE) adds enterprise APIs such as Servlets, JPA and Bean Validation, now in the `jakarta.*` namespace.
- **Release model**: a new feature release every 6 months (March and September). LTS releases are 8, 11, 17, 21 and 25. Most companies run 17 or 21 in production; know what changed in each.
- **JLS vs JEP**: the Java Language Specification defines the language rules. JDK Enhancement Proposals (JEPs) describe new features, e.g. JEP 444 Virtual Threads.
- **Naming conventions**: classes `PascalCase`, methods/variables `camelCase`, constants `UPPER_SNAKE_CASE`, packages lowercase reverse-domain (`com.company.app`).

CODE · COMPILE AND RUN

```
javac Hello.java          // produces Hello.class (bytecode)
java Hello arg1 arg2      // runs on the JVM, args = ["arg1","arg2"]
java Hello.java           // single-file source launch (Java 11+)
jshell                    // REPL for quick experiments
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Why is Java platform-independent while the JVM is platform-dependent?
Q2 What happens if `main` is not declared `static`?
Q3 What is the difference between JDK, JRE and JVM?

TOPIC CHECKLIST · 17 ITEMS

☐ What is Java?	☐ History and features of Java	☐ Java SE vs Jakarta EE
☐ JDK vs JRE vs JVM	☐ Installation and environment setup	☐ Setting JAVA_HOME and PATH
☐ Compiling and running programs	☐ Source code and bytecode	☐ Java program structure
☐ main() method	☐ Command-line arguments	☐ Comments
☐ Naming conventions	☐ Coding standards	☐ JEPs and JLS
☐ Java 8, 11, 17, 21, 25	☐ LTS releases and compatibility	

02 Variables and Data Types

KEY REVISION NOTES

- **Primitive sizes**: `byte` 8-bit, `short` 16, `int` 32, `long` 64, `float` 32, `double` 64, `char` 16-bit unsigned UTF-16 code unit, `boolean` true/false (size is JVM-dependent).
- **Default values**: fields get defaults (0, 0.0, false, `'\u0000'`, `null`). Local variables get no default and must be assigned before use, otherwise compilation fails.
- **Casting**: widening (`int` to `long`, `int` to `double`) is automatic. Narrowing (`double` to `int`) needs an explicit cast and may lose data. Integer overflow wraps around silently: `Integer.MAX_VALUE + 1` is `Integer.MIN_VALUE`.
- **Type promotion**: `byte`, `short` and `char` are promoted to `int` in arithmetic, so `byte b = 1; b = b + 1;` fails to compile while `b += 1;` works because compound assignment includes an implicit cast.
- **Wrapper classes and autoboxing**: `Integer`, `Long`, etc. wrap primitives so they can be used in collections. `Integer` caches -128 to 127, so `Integer a = 127, b = 127; a == b` is true, but with 128 it is false. Always compare wrappers with `equals()`. Unboxing a `null` throws `NullPointerException`.
- **Floating point**: `0.1 + 0.2 != 0.3` because binary floating point cannot represent these exactly. Use `BigDecimal` (created from a String) for money.
- **var (Java 10)**: local variable type inference. The type is fixed at compile time, so Java stays statically typed. Not allowed for fields, method parameters or return types, and `var x = null;` does not compile.

CODE · CLASSIC TRAPS

```
Integer a = 128, b = 128;
System.out.println(a == b);        // false (outside Integer cache)
System.out.println(a.equals(b));   // true

long big = Integer.MAX_VALUE + 1;  // overflow happens in int first!
long ok  = Integer.MAX_VALUE + 1L; // 2147483648

BigDecimal price = new BigDecimal("0.10");  // never new BigDecimal(0.1)
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Why does `Integer == Integer` sometimes return true and sometimes false?

Q2 Where are local variables, instance variables and static variables stored?

Q3 Why should money never be stored as `double`?

TOPIC CHECKLIST · 17 ITEMS

☐ Variables and declarations	☐ Local variables	☐ Instance variables
☐ Static/class variables	☐ Primitives: byte, short, int, long	☐ Primitives: float, double, char, boolean
☐ Reference data types	☐ Literals	☐ Constants using final
☐ Type casting	☐ Widening and narrowing	☐ Type promotion
☐ Default values	☐ Scope and lifetime	☐ Wrapper classes
☐ Autoboxing and unboxing	☐ var type inference	

03 Operators

KEY REVISION NOTES

- **Arithmetic**: integer division truncates toward zero (`7 / 2 == 3`). The `%` result takes the sign of the dividend (`-7 % 3 == -1`). Division by zero throws `ArithmeticException` for integers but gives `Infinity` or `NaN` for doubles.
- **Increment**: `i++` returns the old value then increments; `++i` increments then returns the new value.
- **Short-circuit**: `&&` and `||` stop evaluating as soon as the result is known, so `obj != null && obj.isValid()` is safe. `&` and `|` always evaluate both sides.
- **Bitwise and shift**: `&`, `|`, `^`, `~`. `<<` shifts left, `>>` is arithmetic (keeps the sign bit), `>>>` is logical (fills with zeros). `n & (n - 1) == 0` tests a power of two; `x ^ x == 0` is used in XOR tricks.
- **Ternary**: `cond ? a : b` is an expression and returns a value. Mixed types are promoted (`true ? 1 : 2.0` gives `1.0`).
- **instanceof with pattern matching (Java 16)**: `if (obj instanceof String s)` tests and casts in one step; `s` is in scope only where the test is true.
- **Precedence**: postfix, unary, multiplicative, additive, shift, relational, equality, bitwise, logical, ternary, assignment. When in doubt, use parentheses for readability.

CODE · PREDICT THE OUTPUT

```
int i = 5;
i = i++ + ++i;      // 5 + 7 = 12
System.out.println(-7 % 3);     // -1
System.out.println(-8 >> 1);    // -4
System.out.println(-8 >>> 28);  // 15
Object o = "hi";
if (o instanceof String s && s.length() > 1) { System.out.println(s); }
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the difference between `>>` and `>>>`?

Q2 Why is `&&` preferred over `&` for boolean conditions?

Q3 What does `i = i++;` do to the value of `i`?

TOPIC CHECKLIST · 13 ITEMS

☐ Arithmetic operators	☐ Assignment operators	☐ Relational operators
☐ Logical operators	☐ Unary operators	☐ Increment and decrement
☐ Bitwise operators	☐ Shift operators	☐ Ternary operator
☐ instanceof	☐ Operator precedence	☐ Short-circuit evaluation
☐ Type conversion in expressions		

04 Control Flow Statements

KEY REVISION NOTES

- **switch types**: classic switch works on `byte`, `short`, `char`, `int` (and their wrappers), `String` and `enum`. Java 21 pattern matching for switch works on any reference type, including `case null`.
- **Fall-through**: in a classic `case X:` switch, execution falls into the next case unless you `break`. Forgetting `break` is a common bug.
- **Switch expressions (Java 14)**: the arrow form `case X -> value` has no fall-through, returns a value, uses `yield` inside a block, and must be exhaustive (cover every case or have a `default`).
- **Loops**: `for` when the count is known, `while` when the condition is checked first, `do-while` when the body must run at least once. The enhanced `for` cannot access the index, and removing elements while iterating a collection throws `ConcurrentModificationException`.
- **Jumping**: `break` exits the innermost loop or switch, `continue` skips to the next iteration, `return` leaves the method. Labeled `break outer;` exits an outer loop directly, which is the clean way to leave nested loops.

CODE · SWITCH EXPRESSION AND LABELED BREAK

```
String type = switch (day) {
    case SATURDAY, SUNDAY -> "Weekend";
    case MONDAY -> { log("start"); yield "Weekday"; }
    default -> "Weekday";
};

outer:
for (int[] row : grid)
    for (int v : row)
        if (v == target) break outer;
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the difference between a switch statement and a switch expression?
Q2 How do you exit two nested loops at once?
Q3 When would you choose `do-while` over `while`?

TOPIC CHECKLIST · 15 ITEMS

☐ if	☐ if-else	☐ else-if
☐ Nested conditions	☐ switch	☐ Switch expressions
☐ for loop	☐ Enhanced for loop	☐ while loop
☐ do-while loop	☐ Nested loops	☐ break
☐ continue	☐ return	☐ Labeled break and continue

05 Arrays

KEY REVISION NOTES

- **Basics**: arrays are objects of fixed size, zero-indexed, with a `length` field (not a method). Elements are initialised to default values. Invalid indexes throw `ArrayIndexOutOfBoundsException`.
- **Multidimensional and jagged**: `int[][] m = new int[3][4]` is an array of arrays. Rows can have different lengths (`new int[3][]`), which is a jagged array.
- **Copying**: `Arrays.copyOf`, `Arrays.copyOfRange`, `System.arraycopy` and `clone()` all make shallow copies; for arrays of objects the elements themselves are shared.
- **Comparison**: `==` compares references. `Arrays.equals(a, b)` compares elements. `Arrays.deepEquals` is needed for nested arrays. Print with `Arrays.toString` or `Arrays.deepToString`.
- **Sorting and searching**: `Arrays.sort` uses dual-pivot quicksort for primitives and TimSort (stable) for objects. `Arrays.binarySearch` requires a sorted array, otherwise the result is undefined.

- **Varargs**: `int sum(int... nums)` is received as an array. Only one varargs parameter is allowed and it must be last.
- **Arrays vs collections**: arrays are fixed-size, hold primitives and are covariant (`String[]` is an `Object[]`, which can cause `ArrayStoreException`). Collections are resizable, hold objects only, and generics are invariant and type-safe.

CODE · COMMON OPERATIONS

```
int[] a = {5, 2, 9};
Arrays.sort(a);                          // [2, 5, 9]
int idx = Arrays.binarySearch(a, 9);     // 2
int[] b = Arrays.copyOf(a, 5);           // [2, 5, 9, 0, 0]
int[][] jagged = { {1}, {2, 3}, {4, 5, 6} };
System.out.println(Arrays.deepToString(jagged));
List<Integer> list = Arrays.stream(a).boxed().toList();
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the difference between `length`, `length()` and `size()`?
Q2 Why can arrays throw `ArrayStoreException` while generic lists cannot?
Q3 Is `clone()` on an array a deep or shallow copy?

TOPIC CHECKLIST · 16 ITEMS

☐ One-dimensional arrays	☐ Multidimensional arrays	☐ Declaration and initialization
☐ Array traversal	☐ Array copying	☐ Array comparison
☐ Anonymous arrays	☐ Jagged arrays	☐ Arrays utility class
☐ Sorting and searching	☐ Arrays.equals()	☐ Arrays.deepEquals()
☐ Arrays.copyOf()	☐ Arrays.binarySearch()	☐ Varargs
☐ Arrays vs collections		

06 Strings

KEY REVISION NOTES

- **Immutability**: a `String` never changes after creation. This makes strings thread-safe, safe as `HashMap` keys (hash code can be cached), enables the string pool, and prevents security issues with class names, URLs and file paths. Methods like `toUpperCase()` return new objects.
- **String pool**: literals are interned in the pool (on the heap since Java 7), so `"a" == "a"` is true. `new String("a")` always creates a new heap object; `intern()` returns the pooled copy.
- **Comparison**: `==` compares references; `equals()` compares characters; `equalsIgnoreCase()` ignores case; `compareTo()` orders lexicographically. Write `"literal".equals(var)` to avoid NPEs.
- **StringBuilder vs StringBuffer**: both are mutable. `StringBuilder` is not synchronised and is the default choice; `StringBuffer` is synchronised and legacy. Use a builder when concatenating in loops, because `+` in a loop creates a new String each time.
- **Useful methods**: `substring(begin, end)` (end exclusive), `charAt`, `indexOf`, `contains`, `replace`, `startsWith`, `isBlank`, `repeat`, `lines`, `chars()`. `strip()` is Unicode-aware while `trim()` only removes characters up to U+0020.
- **split() takes a regex**: `"a.b".split(".")` returns an empty array; use `split("\\.")` or `Pattern.quote(".")`.
- **Text blocks (Java 15)**: `"""` multi-line strings for JSON, SQL and HTML. Formatting with `String.format` or `"%s".formatted(x)`.
- **Regex**: compile a `Pattern` once and reuse it (it is thread-safe); `Matcher` is not thread-safe. Groups with `matcher.group(1)`.

CODE · STRING POOL AND BUILDER

```
String s1 = "java";
String s2 = "java";
String s3 = new String("java");
s1 == s2;            // true  (same pooled object)
s1 == s3;            // false (different object)
s1 == s3.intern();   // true

StringBuilder sb = new StringBuilder();
for (String w : words) sb.append(w).append(',');
String csv = sb.toString();
```

INTERVIEW CHECK · can you answer these out loud?

Q1 How many objects can `new String("hello")` create?
Q2 Why is String immutable and final?
Q3 Why is `char[]` sometimes preferred over String for passwords?

TOPIC CHECKLIST · 24 ITEMS

☐ String class	☐ String literals	☐ String pool
☐ String immutability	☐ Concatenation	☐ Comparison
☐ == vs equals()	☐ equalsIgnoreCase()	☐ Indexing and extraction
☐ substring()	☐ charAt()	☐ indexOf()
☐ contains()	☐ split()	☐ replace()
☐ trim() vs strip()	☐ startsWith() / endsWith()	☐ StringBuilder
☐ StringBuffer	☐ String formatting	☐ Text blocks
☐ Unicode and encoding	☐ Regular expressions	☐ Pattern and Matcher

07 Object-Oriented Programming (OOP)

KEY REVISION NOTES

- **Encapsulation**: keep fields `private` and expose behaviour through methods, so the class protects its own invariants (e.g. balance never negative).
- **Abstraction**: expose what an object does and hide how, using interfaces and abstract classes.
- **Inheritance**: an IS-A relationship via `extends`. Java allows single class inheritance but multiple interface implementation. `super` calls the parent constructor or method.
- **Polymorphism**: overloading is compile-time (same name, different parameters). Overriding is runtime: the JVM picks the method of the actual object type (dynamic method dispatch).
- **Overriding rules**: same signature, covariant return type allowed, cannot reduce visibility, cannot throw broader checked exceptions. Static methods are hidden, not overridden. Private and final methods cannot be overridden. Fields are not polymorphic.
- **Abstract class vs interface**: an abstract class can have state, constructors and any access level; use it for shared base implementation. An interface is a contract; a class can implement many. Since Java 8 interfaces have `default` and `static` methods, and since Java 9 `private` methods. If two interfaces provide the same default method, the class must override it (diamond problem).
- **Composition over inheritance**: prefer HAS-A (holding a reference) for flexibility; inheritance couples the subclass to parent internals. Aggregation means the part can exist independently; composition means the part's lifecycle is owned by the whole.
- **Coupling and cohesion**: aim for low coupling (few dependencies between classes) and high cohesion (a class does one focused job).
- **Initialization order**: static blocks (once, on class load), then instance initializer blocks, then the constructor body. Constructor chaining with `this(...)` or `super(...)` must be the first statement.
- **Immutable class recipe**: `final` class, `private final` fields, no setters, initialise everything in the constructor, make defensive copies of mutable inputs and outputs.

- **Nested classes**: static nested (no outer instance), inner (holds a reference to the outer instance, which can leak memory), local, and anonymous classes.
- **Modern types**: `enum` (fixed set of constants, can have fields and methods, ideal for singletons), `record` (immutable data carrier with generated constructor, accessors, `equals`, `hashCode`, `toString`), and `sealed` classes that restrict which classes can extend them, enabling exhaustive switches.

CODE · RECORDS + SEALED TYPES + PATTERN MATCHING

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {
    Circle {                        // compact constructor for validation
        if (r <= 0) throw new IllegalArgumentException("r > 0");
    }
}
record Square(double side) implements Shape {}

double area(Shape s) {
    return switch (s) {             // exhaustive, no default needed
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.side() * q.side();
    };
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Can you override a static or private method? Explain.
Q2 Abstract class vs interface since Java 8: when would you use each?
Q3 How do you create an immutable class? Why would you?
Q4 Explain composition vs inheritance with an example.

TOPIC CHECKLIST · 37 ITEMS

☐ Class declaration	☐ Object creation	☐ Fields and methods
☐ Constructors	☐ Default and parameterized constructors	☐ Constructor overloading
☐ this keyword	☐ Instance and static members	☐ Instance initialization blocks
☐ Static initialization blocks	☐ Encapsulation	☐ Abstraction
☐ Inheritance	☐ Polymorphism	☐ Method overloading
☐ Method overriding	☐ Dynamic method dispatch	☐ Upcasting and downcasting
☐ super keyword	☐ final classes, methods, variables	☐ Abstract classes
☐ Interfaces	☐ Default and static interface methods	☐ Private interface methods
☐ Multiple inheritance via interfaces	☐ Composition vs inheritance	☐ Association, aggregation, composition
☐ Coupling and cohesion	☐ Object equality and identity	☐ Immutable classes
☐ Nested classes	☐ Inner classes	☐ Local classes
☐ Anonymous classes	☐ Enum classes	☐ Records
☐ Sealed classes and interfaces		

08 Access Modifiers and Packages

KEY REVISION NOTES

- **Top-level classes** can only be `public` or package-private. Only one public top-level class per file, and the file name must match it.
- **protected across packages**: a subclass in another package can access a protected member only through its own inheritance (`this.x` or a subclass reference), not via an arbitrary parent-type reference.
- **Imports**: `import java.util.List;` saves typing; it does not load code. `import static java.lang.Math.*;` imports static members. A fully qualified name (`java.util.Date`) resolves name clashes.
- **JAR files**: a ZIP of class files and resources with a `META-INF/MANIFEST.MF`. An executable JAR names the `Main-Class`.
- **Module system (JPMS, Java 9)**: `module-info.java` declares `requires` (dependencies), `exports` (packages whose public types are accessible at compile and run time) and `opens` (packages available for deep reflection at run time, which frameworks like Hibernate and Jackson need). Modules provide strong encapsulation.
- **Classpath vs module path**: the classpath is a flat list of JARs with no encapsulation and possible duplicate classes (JAR hell). The module path enforces module boundaries and reliable configuration.

Modifier	Class	Package	Subclass (other pkg)	World
private	Yes	No	No	No
(default) package-private	Yes	Yes	No	No
protected	Yes	Yes	Yes (via inheritance)	No
public	Yes	Yes	Yes	Yes

CODE · MODULE-INFO.JAVA

```
module com.shop.orders {
    requires java.sql;
    requires com.shop.common;
    exports com.shop.orders.api;          // public API
    opens com.shop.orders.model;          // for reflection (JPA/Jackson)
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Can a top-level class be `private` or `protected`?
Q2 What is the difference between `exports` and `opens`?
Q3 What is package-private access and when is it useful?

TOPIC CHECKLIST · 15 ITEMS

☐ public	☐ private	☐ protected
☐ Package-private access	☐ Access across packages	☐ Creating packages
☐ Import statements	☐ Static imports	☐ Fully qualified names
☐ Package naming conventions	☐ JAR files	☐ Java module system
☐ module-info.java	☐ requires, exports, opens	☐ Classpath vs module path

09 Exception Handling

KEY REVISION NOTES

- **Hierarchy**: `Throwable` splits into `Error` (serious JVM problems such as `OutOfMemoryError` and `StackOverflowError`; do not catch) and `Exception`. `RuntimeException` and its subclasses are unchecked; all other exceptions are checked.
- **Checked vs unchecked**: checked exceptions (`IOException`, `SQLException`) must be caught or declared with `throws`; they represent recoverable external conditions. Unchecked exceptions (`NullPointerException`, `IllegalArgumentException`, `IllegalStateException`) usually indicate programming bugs. Modern frameworks such as Spring favour unchecked exceptions.
- **finally**: always runs after try/catch, except on `System.exit()` or a JVM crash. A `return` in `finally` overrides any earlier return or exception, so never do it.
- **throw vs throws**: `throw` raises an exception instance; `throws` declares in the method signature that it may propagate.
- **Try-with-resources (Java 7)**: closes any `AutoCloseable` automatically, in reverse order of declaration. If `close()` also throws, that exception is added as suppressed (`e.getSuppressed()`) instead of hiding the original.
- **Exception chaining**: wrap a low-level exception while keeping the cause: `throw new OrderException("Failed to save", e);`. Never lose the stack trace.
- **Catch order and multi-catch**: catch the most specific exceptions first, otherwise compilation fails. `catch (IOException | SQLException e)` handles several types with one block.
- **Propagation**: an uncaught exception unwinds the call stack frame by frame until a matching handler is found, or the thread terminates.
- **Best practices**: don't swallow exceptions with empty catch blocks; don't catch `Exception` or `Throwable` broadly; either log or rethrow, not both (avoid duplicate logs); create meaningful custom exceptions; fail fast with validation at boundaries.

CODE · CUSTOM EXCEPTION + TRY-WITH-RESOURCES

```
public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String msg) { super(msg); }
}

try (var reader = Files.newBufferedReader(path);
     var writer = Files.newBufferedWriter(out)) {
    reader.lines().forEach(l -> write(writer, l));
} catch (IOException e) {
    throw new UncheckedIOException("Copy failed: " + path, e);
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Difference between `final`, `finally` and `finalize`?
Q2 What happens if an exception is thrown in both try and finally?
Q3 Should custom exceptions be checked or unchecked? Justify.

TOPIC CHECKLIST · 20 ITEMS

☐ Errors vs exceptions	☐ Exception hierarchy	☐ Checked exceptions
☐ Unchecked exceptions	☐ try	☐ catch
☐ Multiple catch blocks	☐ Multi-catch	☐ finally
☐ throw	☐ throws	☐ Custom exceptions
☐ Exception propagation	☐ Nested try blocks	☐ Try-with-resources
☐ Suppressed exceptions	☐ Exception chaining	☐ Best practices
☐ Error vs Exception	☐ Fail-fast vs fail-safe handling	

10 Object Class and Core Methods

KEY REVISION NOTES

- **Object** is the root of every class. Key methods: `toString`, `equals`, `hashCode`, `getClass`, `clone`, `wait`, `notify`, `notifyAll`, and the deprecated `finalize`.
- **equals contract**: reflexive, symmetric, transitive, consistent, and `x.equals(null)` is false. Use `getClass()` checks for strict equality or `instanceof` (pattern) when subclasses may be equal.
- **hashCode contract**: if two objects are equal they must have the same hash code; unequal objects may share one (a collision). Always override `equals` and `hashCode` together, otherwise objects get lost in `HashMap` and `HashSet`. Use only immutable fields.
- **Objects utility**: `Objects.equals(a, b)` (null-safe), `Objects.hash(...)`, `Objects.requireNonNull(x, "msg")`, `Objects.toString(x, "default")`.
- **clone()**: requires the `Cloneable` marker interface, produces a shallow copy by default, and is widely considered broken. Prefer copy constructors or static factory methods.
- **wait/notify**: must be called while holding the object's monitor (inside `synchronized`), otherwise `IllegalMonitorStateException`. `wait()` releases the lock; always call it in a loop that re-checks the condition (spurious wake-ups).
- **finalize()**: deprecated since Java 9 and deprecated for removal since Java 18; unpredictable and slow. Use try-with-resources or `java.lang.ref.Cleaner`.
- **Comparable vs Comparator**: `Comparable.compareTo` defines natural ordering inside the class (one ordering). `Comparator` is external and you can define many. Keep `compareTo` consistent with `equals` for `TreeSet`/`TreeMap`.

CODE · EQUALS/HASHCODE AND COMPARATORS

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee e)) return false;
    return id == e.id && Objects.equals(email, e.email);
}
@Override public int hashCode() { return Objects.hash(id, email); }

list.sort(Comparator.comparing(Employee::dept)
                    .thenComparing(Employee::salary, Comparator.reverseOrder()));
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What happens if you override `equals()` but not `hashCode()`?
Q2 Comparable vs Comparator: when do you use each?
Q3 Why must `wait()` be called inside a synchronized block and in a loop?

TOPIC CHECKLIST · 15 ITEMS

☐ Object class	☐ toString()	☐ equals()
☐ hashCode()	☐ getClass()	☐ clone()
☐ wait()	☐ notify()	☐ notifyAll()
☐ finalize() history and deprecation	☐ Objects utility class	☐ Equality and hashing contracts
☐ Comparable	☐ Comparator	☐ Natural vs custom ordering

11 Java Collections Framework

KEY REVISION NOTES

- **Hierarchy**: `Iterable` → `Collection` → `List`, `Set`, `Queue` → `Deque`. `Map` is separate and does not extend `Collection`.
- **ArrayList vs LinkedList**: ArrayList is a resizable array with fast random access and is the right default. LinkedList is rarely better in practice; use `ArrayDeque` for queue and stack behaviour.
- **HashMap internals**: an array of buckets (default capacity 16, load factor 0.75). The index is `(h ^ (h >>> 16)) & (n - 1)`. Collisions form a linked list, which becomes a red-black tree when a bucket reaches 8 entries and capacity is at least 64. When size exceeds capacity × load factor, the table doubles and entries are redistributed. Allows one `null` key.
- **Mutable keys are dangerous**: if a key's fields change after insertion, its hash changes and the entry becomes unreachable. Use immutable keys (String, records).
- **Sets**: `HashSet` is backed by a `HashMap`; `LinkedHashSet` keeps insertion order; `TreeSet` is sorted; `EnumSet` is a very fast bit vector for enums.
- **Legacy synchronised classes**: `Vector`, `Stack` and `Hashtable` lock every method and are slow. Use `ArrayList`, `ArrayDeque` and `ConcurrentHashMap`. `Hashtable` and `ConcurrentHashMap` do not allow null keys or values.
- **Special maps**: `WeakHashMap` drops entries whose keys are no longer strongly referenced (caches); `IdentityHashMap` uses `==` instead of `equals`; `EnumMap` is array-backed for enum keys.
- **Fail-fast vs fail-safe**: iterators of standard collections track `modCount` and throw `ConcurrentModificationException` if the collection changes during iteration. Remove safely with `Iterator.remove()` or `removeIf()`. Concurrent collections (`CopyOnWriteArrayList`, `ConcurrentHashMap`) have weakly consistent iterators that never throw.
- **Immutable vs unmodifiable**: `List.of()` and friends are truly immutable and reject nulls. `Collections.unmodifiableList(list)` is a read-only view; changes to the original still show through. `Arrays.asList()` is fixed-size and backed by the array (`set` works, `add` throws).
- **Choosing**: need order and index access → `ArrayList`; uniqueness → `HashSet`; sorted → `TreeSet`/`TreeMap`; key-value lookup → `HashMap`; FIFO/LIFO → `ArrayDeque`; priority → `PriorityQueue`; LRU cache → `LinkedHashMap` with access order.

Structure	Get / Access	Insert	Remove	Contains / Search	Notes
ArrayList	O(1)	O(1)* at end	O(n)	O(n)	*amortised; grows ~1.5x
LinkedList	O(n)	O(1) at ends	O(1) at ends	O(n)	Doubly linked; poor cache locality
ArrayDeque	O(1) ends	O(1)*	O(1) ends	O(n)	Best stack/queue; no nulls
HashMap / HashSet	O(1) avg	O(1) avg	O(1) avg	O(1) avg	O(log n) worst in treeified bucket
LinkedHashMap	O(1)	O(1)	O(1)	O(1)	Insertion or access order (LRU)
TreeMap / TreeSet	O(log n)	O(log n)	O(log n)	O(log n)	Red-black tree; sorted, range ops
PriorityQueue	O(1) peek	O(log n)	O(log n) poll	O(n)	Binary min-heap by default

CODE · LRU CACHE IN A FEW LINES

```
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LruCache(int capacity) {
        super(16, 0.75f, true);           // true = access order
        this.capacity = capacity;
    }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) {
        return size() > capacity;
    }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 How does HashMap work internally? What changed in Java 8?
Q2 HashMap vs Hashtable vs ConcurrentHashMap vs synchronizedMap?
Q3 How do you avoid ConcurrentModificationException?
Q4 Design an LRU cache.

TOPIC CHECKLIST · 42 ITEMS

☐ Iterable	☐ Collection	☐ List
☐ Set	☐ Queue	☐ Deque
☐ Map	☐ ArrayList	☐ LinkedList
☐ Vector	☐ Stack	☐ HashSet
☐ LinkedHashSet	☐ TreeSet	☐ EnumSet
☐ PriorityQueue	☐ ArrayDeque	☐ Blocking queues
☐ HashMap	☐ LinkedHashMap	☐ TreeMap
☐ Hashtable	☐ ConcurrentHashMap	☐ WeakHashMap
☐ IdentityHashMap	☐ EnumMap	☐ Iterators and list iterators
☐ Enhanced for loop	☐ Fail-fast behavior	☐ Iterator.remove()
☐ Sorting collections	☐ Natural ordering	☐ Custom comparators
☐ Collections utility class	☐ Immutable collections	☐ Unmodifiable views
☐ Synchronized collections	☐ Arrays.asList()	☐ List.of(), Set.of(), Map.of()
☐ Generics in collections	☐ Time complexity of operations	☐ Choosing the right data structure

12 Generics

KEY REVISION NOTES

- **Why**: compile-time type safety and no casting. Errors move from run time (`ClassCastException`) to compile time.
- **Generic classes and methods**: `class Box<T>`, `static <T> T first(List<T> list)`. The type parameter of a generic method is declared before the return type.
- **Bounded types**: `<T extends Comparable<T>>` lets you call `compareTo`. Multiple bounds: `<T extends Number & Comparable<T>>` (class first, then interfaces).
- **Invariance**: `List<Integer>` is not a subtype of `List<Number>`, even though `Integer` is a `Number`. Otherwise you could add a `Double` into a list of Integers.
- **Wildcards**: `List<? extends Number>` can be read as Number but you cannot add (except null). `List<? super Integer>` accepts Integers but reads only as Object. `List<?>` is a list of unknown type.
- **PECS**: Producer Extends, Consumer Super. If a parameter produces values for you, use `extends`; if it consumes values you put in, use `super`.
- **Type erasure**: generic type information is removed at compile time and replaced by bounds or `Object`, with casts inserted. Consequences: no `new T()`, no `instanceof List<String>`, no generic arrays (`new T[10]`), no primitives as type arguments, no static fields of type `T`, and overloads differing only by type argument clash.
- **Raw types**: `List list = new ArrayList();` disables type checking. Exists only for backward compatibility; avoid.

CODE · PECS IN ACTION

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (T item : src) dest.add(item);     // read from producer, write to consumer
}

public static double sum(Collection<? extends Number> nums) {
    return nums.stream().mapToDouble(Number::doubleValue).sum();
}

public static <T extends Comparable<T>> T max(List<T> list) { ... }
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is type erasure and what are its consequences?
Q2 Explain PECS with an example.
Q3 What is the difference between `List<?>`, `List<Object>` and a raw `List`?

TOPIC CHECKLIST · 17 ITEMS

☐ Generic classes	☐ Generic methods	☐ Generic interfaces
☐ Type parameters	☐ Bounded type parameters	☐ Multiple bounds
☐ Wildcards	☐ Upper-bounded wildcards	☐ Lower-bounded wildcards
☐ Unbounded wildcards	☐ PECS principle	☐ Type erasure
☐ Raw types	☐ Generic invariance	☐ Generic constructors
☐ Generic restrictions	☐ Generic methods vs overloading	

13 Functional Programming

KEY REVISION NOTES

- **Functional interface**: an interface with exactly one abstract method (SAM). `@FunctionalInterface` makes the compiler enforce it. Methods inherited from `Object` (like `equals` in `Comparator`) do not count.
- **Lambdas**: `(params) -> expression` or `-> { block }`. A lambda is an instance of a functional interface, compiled using `invokedynamic` rather than an anonymous class.
- **Effectively final capture**: lambdas can use local variables only if they are never reassigned. This avoids data races and confusing semantics, since the lambda may run later or on another thread.
- **Core interfaces**: `Predicate<T>` (test → boolean), `Function<T,R>` (apply), `Consumer<T>` (accept, no result), `Supplier<T>` (get, no input), `UnaryOperator<T>` (T → T), `BinaryOperator<T>` ((T,T) → T), plus `Bi*` versions and primitive versions like `IntPredicate`.
- **Method references**: static (`Integer::parseInt`), bound instance (`System.out::println`), unbound instance (`String::toUpperCase`, the first argument becomes the receiver) and constructor (`ArrayList::new`).
- **Composition**: `f.andThen(g)` applies f then g; `f.compose(g)` applies g then f; `p.and(q)`, `p.or(q)`, `p.negate()`, `Predicate.not(String::isBlank)`.
- **Higher-order functions**: functions that take or return functions, e.g. `Comparator.comparing(keyExtractor)` or a method returning a configured `Predicate`.

CODE · COMPOSING BEHAVIOUR

```
Predicate<String> notBlank = Predicate.not(String::isBlank);
Predicate<String> shortWord = s -> s.length() < 5;
Function<String, String> trim = String::strip;
Function<String, Integer> len = trim.andThen(String::length);

Supplier<List<String>> factory = ArrayList::new;
BinaryOperator<Integer> add = Integer::sum;
words.stream().filter(notBlank.and(shortWord)).forEach(System.out::println);
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Why must variables captured by a lambda be effectively final?
Q2 Is `Comparator` a functional interface even though it has many methods?
Q3 Name the four kinds of method references.

TOPIC CHECKLIST · 15 ITEMS

☐ Functional programming concepts	☐ Lambda expressions	☐ Functional interfaces
☐ @FunctionalInterface	☐ Method references	☐ Constructor references
☐ Predicate	☐ Function	☐ Consumer
☐ Supplier	☐ UnaryOperator	☐ BinaryOperator
☐ Function composition	☐ Closures and effectively final	☐ Higher-order operations

14 Stream API

KEY REVISION NOTES

- **Pipeline**: source (collection, array, `Stream.of`, `Files.lines`, `IntStream.range`) → zero or more intermediate operations → exactly one terminal operation. Streams do not store data and do not modify the source.
- **Laziness**: intermediate operations (`filter`, `map`, `sorted`...) build a recipe and run only when a terminal operation (`collect`, `forEach`, `reduce`, `count`...) is called. Elements flow one at a time through the whole pipeline.
- **Short-circuiting**: `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch` and `limit` can stop early, even on infinite streams.
- **No reuse**: a stream can be consumed only once; a second terminal operation throws `IllegalStateException`.
- **map vs flatMap**: `map` transforms each element one-to-one. `flatMap` maps each element to a stream and flattens them (e.g. a list of orders into a stream of all their items).
- **Collectors**: `toList()`, `toSet()`, `joining(", ")`, `groupingBy(classifier, downstream)`, `partitioningBy(predicate)`, `counting()`, `summingInt`, `averagingDouble`, `mapping`. `toMap` throws `IllegalStateException` on duplicate keys unless you give a merge function.
- **toList() vs Collectors.toList()**: `Stream.toList()` (Java 16) returns an unmodifiable list; `Collectors.toList()` returns a mutable list (implementation unspecified).
- **Primitive streams**: `IntStream`, `LongStream`, `DoubleStream` avoid boxing and offer `sum()`, `average()`, `summaryStatistics()`. Convert with `mapToInt` and back with `boxed()`.
- **peek and side effects**: `peek` is for debugging only. Avoid modifying external state inside streams.
- **Parallel streams**: split work over the common `ForkJoinPool`. Helpful for large, CPU-bound, stateless work with easily splittable sources (arrays, ArrayList). Harmful for small data, blocking I/O, ordered operations, or shared mutable state.
- **Streams vs loops**: streams are declarative and composable; loops are better for complex control flow, early returns with side effects, or checked exceptions. Performance is usually similar for simple cases.

CODE · GROUPING AND AGGREGATION

```
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::dept, Collectors.counting()));

Map<String, Double> avgSalary = employees.stream()
    .collect(Collectors.groupingBy(Employee::dept,
             Collectors.averagingDouble(Employee::salary)));

Map<Boolean, List<Employee>> seniors = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.age() > 40));

List<String> skills = employees.stream()
    .flatMap(e -> e.skills().stream()).distinct().sorted().toList();

Map<Long, Employee> byId = employees.stream()
    .collect(Collectors.toMap(Employee::id, e -> e, (a, b) -> a));
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the difference between intermediate and terminal operations?
Q2 map vs flatMap with an example.
Q3 Find the second highest salary using streams.
Q4 When are parallel streams a bad idea?

TOPIC CHECKLIST · 32 ITEMS

☐ Creating streams	☐ Intermediate vs terminal	☐ filter()
☐ map()	☐ flatMap()	☐ distinct()
☐ sorted()	☐ limit()	☐ skip()
☐ peek()	☐ forEach()	☐ reduce()
☐ collect()	☐ Collectors.toList()	☐ Collectors.toMap()
☐ groupingBy()	☐ partitioningBy()	☐ joining()
☐ count()	☐ findFirst()	☐ findAny()
☐ anyMatch()	☐ allMatch()	☐ noneMatch()
☐ Primitive streams	☐ IntStream, LongStream, DoubleStream	☐ Optional results
☐ Stream laziness	☐ Stream reuse limitations	☐ Parallel streams
☐ Stream performance	☐ Stream vs loops	

15 Optional API

KEY REVISION NOTES

- **Purpose**: a container that may or may not hold a value, designed mainly as a return type to make "no result" explicit and avoid returning `null`.
- **Creating**: `Optional.of(x)` throws NPE if x is null; `Optional.ofNullable(x)` is safe; `Optional.empty()` is the empty instance.
- **Transforming**: `map` transforms a present value; `flatMap` is used when the mapping function itself returns an `Optional`; `filter` keeps the value only if it matches.
- **orElse vs orElseGet**: `orElse(createDefault())` always evaluates its argument, even when a value is present. `orElseGet(() -> createDefault())` is lazy. Use `orElseGet` for anything expensive (DB calls, object creation).
- **orElseThrow**: `orElseThrow()` throws `NoSuchElementException`; `orElseThrow(() -> new UserNotFoundException(id))` throws your own exception, which is the common pattern in services.
- **Anti-patterns**: calling `get()` without checking; `if (opt.isPresent()) opt.get()` instead of `ifPresent`/`map`; Optional as a field, method parameter or collection element; `Optional<List<T>>` (return an empty list instead); Optional is also not `Serializable`.

CODE · IDIOMATIC USAGE

```
public UserDto getEmail(long id) {
    return userRepository.findById(id)           // Optional<User>
        .filter(User::isActive)
        .map(UserDto::from)
        .orElseThrow(() -> new UserNotFoundException(id));
}

String city = user.flatMap(User::address)       // address() returns Optional
                  .map(Address::city)
                  .orElse("Unknown");
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the difference between `orElse` and `orElseGet`?
Q2 Why shouldn't Optional be used for fields or parameters?
Q3 map vs flatMap on Optional?

TOPIC CHECKLIST · 15 ITEMS

☐ Creating Optional	☐ Optional.of()	☐ Optional.ofNullable()
☐ Optional.empty()	☐ isPresent()	☐ ifPresent()
☐ map()	☐ flatMap()	☐ filter()
☐ orElse()	☐ orElseGet()	☐ orElseThrow()
☐ Avoiding improper usage	☐ Optional in return types	☐ Optional vs null

16 Date and Time API

KEY REVISION NOTES

- **Legacy problems**: `java.util.Date` and `Calendar` are mutable, not thread-safe, have 0-based months and a confusing API. `SimpleDateFormat` is not thread-safe.
- **java.time (Java 8)**: all classes are immutable and thread-safe, including `DateTimeFormatter`. Methods like `plusDays` return new objects.
- **Which class?** `LocalDate`, `LocalTime`, `LocalDateTime` have no time zone (wall-clock values such as a birthday). `ZonedDateTime` has a region zone with DST rules (`Asia/Kolkata`, `Europe/London`). `OffsetDateTime` has a fixed offset (`+05:30`) and is good for APIs and databases. `Instant` is a point on the UTC timeline, ideal for timestamps and logs.
- **Duration vs Period**: `Duration` is time-based (hours, minutes, seconds, nanos) and works with `Instant`/`LocalTime`. `Period` is date-based (years, months, days) and works with `LocalDate`.
- **Best practice**: store and transmit timestamps in UTC (`Instant` or `OffsetDateTime`), convert to the user's zone only for display. Never rely on the server's default time zone.
- **DST**: adding 1 day to a `ZonedDateTime` keeps local time; adding 24 hours may not. Some local times do not exist (spring forward) or occur twice (fall back).
- **Legacy conversion**: `date.toInstant()`, `Date.from(instant)`, `instant.atZone(zoneId)`.

CODE · TYPICAL OPERATIONS

```
LocalDate today = LocalDate.now();
LocalDate due = today.plusMonths(1).withDayOfMonth(1);
long days = ChronoUnit.DAYS.between(today, due);
Period age = Period.between(birthDate, today);

Instant start = Instant.now();
Duration took = Duration.between(start, Instant.now());

var fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
ZonedDateTime ist = Instant.now().atZone(ZoneId.of("Asia/Kolkata"));
String text = ist.format(fmt);
```

INTERVIEW CHECK · can you answer these out loud?

Q1 LocalDateTime vs ZonedDateTime vs Instant: when do you use each?
Q2 Duration vs Period?
Q3 Why was the java.time API introduced?

TOPIC CHECKLIST · 17 ITEMS

☐ Legacy Date and Calendar	☐ java.time package	☐ LocalDate
☐ LocalTime	☐ LocalDateTime	☐ ZonedDateTime
☐ OffsetDateTime	☐ Instant	☐ Duration
☐ Period	☐ DateTimeFormatter	☐ Parsing and formatting
☐ Time zones	☐ UTC	☐ Date calculations
☐ Daylight-saving time	☐ Legacy date conversion	

17 File Handling and I/O

KEY REVISION NOTES

- **Byte vs character streams**: `InputStream`/`OutputStream` handle raw bytes (images, binary). `Reader`/`Writer` handle characters and need a charset. `InputStreamReader` bridges bytes to characters (Adapter pattern).
- **Buffering**: `BufferedReader`/`BufferedWriter`/`BufferedInputStream` reduce costly system calls by reading and writing in chunks (Decorator pattern). `BufferedReader.readLine()` reads line by line.
- **Charset**: always specify `StandardCharsets.UTF_8`. UTF-8 is the default charset since Java 18 (JEP 400), but explicit is safer for older runtimes.
- **Serialization**: converts an object graph to bytes. Requires the `Serializable` marker interface. `transient` fields are skipped (restored as defaults). Declare `serialVersionUID` so class changes don't break deserialisation with `InvalidClassException`. Deserialising untrusted data is a major security risk; prefer JSON (Jackson) or Protobuf.
- **NIO.2 (java.nio.file)**: `Path.of("data", "a.txt")`, `Files.readString`, `Files.readAllLines`, `Files.write`, `Files.copy`, `Files.move`, `Files.exists`, `Files.walk`, `Files.list`. Streams returned by `Files.lines`/`walk` hold file handles and must be closed.
- **Channels and buffers**: `FileChannel` and `ByteBuffer` give block-oriented and memory-mapped I/O. Selectors allow non-blocking I/O where one thread handles many connections (the basis of Netty).
- **Other**: file attributes via `Files.readAttributes`, locking with `FileChannel.lock()`, directory watching with `WatchService`, and ZIP handling with `ZipInputStream`/`ZipOutputStream` or the zip `FileSystem`.

CODE · MODERN FILE PROCESSING

```
Path input = Path.of("logs", "app.log");
try (Stream<String> lines = Files.lines(input, StandardCharsets.UTF_8)) {
    long errors = lines.filter(l -> l.contains("ERROR")).count();
}

Files.writeString(Path.of("out.txt"), "done", StandardOpenOption.CREATE);

try (Stream<Path> files = Files.walk(Path.of("src"))) {
    files.filter(p -> p.toString().endsWith(".java")).forEach(System.out::println);
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the purpose of `transient` and `serialVersionUID`?
Q2 What are the differences between IO and NIO?
Q3 Why should you close the stream returned by `Files.lines()`?

TOPIC CHECKLIST · 30 ITEMS

☐ File class	☐ Byte streams	☐ Character streams
☐ InputStream	☐ OutputStream	☐ Reader
☐ Writer	☐ Buffered streams	☐ FileInputStream
☐ FileOutputStream	☐ FileReader	☐ FileWriter
☐ BufferedReader	☐ BufferedWriter	☐ PrintWriter
☐ Scanner	☐ Serialization and deserialization	☐ Serializable
☐ transient	☐ serialVersionUID	☐ Object streams
☐ Java NIO	☐ Path and Paths	☐ Files utility class
☐ File attributes	☐ Directory traversal	☐ Channels and buffers
☐ File locking	☐ WatchService	☐ ZIP file handling

18 Multithreading and Concurrency

KEY REVISION NOTES

- **Process vs thread**: a process has its own memory space; threads within a process share the heap but each has its own stack and program counter.
- **Lifecycle**: NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED. Call `start()` to create a new thread; calling `run()` just runs the code on the current thread.
- **Creating work**: prefer implementing `Runnable` (no result) or `Callable<V>` (returns a value, can throw checked exceptions) and submitting to an `ExecutorService`, rather than extending `Thread`. `Future.get()` blocks until the result is ready.
- **Thread pools**: reuse threads to avoid creation cost and bound concurrency. `ThreadPoolExecutor` parameters: core size, max size, keep-alive, work queue, rejection policy. Always `shutdown()` executors. Unbounded queues can cause OutOfMemoryError.
- **synchronized**: provides mutual exclusion and visibility using the object's intrinsic lock (monitor). It is reentrant. Synchronize on a private final lock object, not on `this` or a String.
- **volatile**: guarantees visibility and ordering of a single variable across threads, but not atomicity. `count++` on a volatile is still a race (read-modify-write). Use `AtomicInteger`/`LongAdder` (lock-free CAS) instead.
- **Problems**: race condition (result depends on timing), deadlock (threads wait on each other forever), starvation (a thread never gets CPU or lock), livelock (threads keep reacting to each other without progress).
- **Deadlock conditions**: mutual exclusion, hold and wait, no preemption, circular wait. Prevent with a global lock ordering, `tryLock` with timeouts, or holding fewer locks.
- **sleep vs wait vs join**: `sleep` pauses but keeps locks; `wait` releases the monitor until notified; `join` waits for another thread to finish.
- **Explicit locks**: `ReentrantLock` adds `tryLock`, timeouts, interruptible locking, fairness and multiple `Condition`s (always unlock in `finally`). `ReadWriteLock` allows many readers or one writer.
- **Synchronizers**: `CountDownLatch` (wait until N events happen, one-shot), `CyclicBarrier` (N threads wait for each other, reusable), `Semaphore` (limit concurrent access to N permits), `BlockingQueue` (producer-consumer).
- **Java Memory Model**: defines when writes by one thread become visible to another. Happens-before rules include: unlock → subsequent lock, volatile write → subsequent read, `Thread.start()` → actions in the thread, thread actions → `join()` returning.
- **CompletableFuture**: compose async work: `supplyAsync`, `thenApply` (map), `thenCompose` (flatMap), `thenCombine`, `allOf`, `exceptionally`, `handle`. Pass a custom executor for I/O work.
- **Fork/Join**: divide-and-conquer with `RecursiveTask`; idle workers steal tasks from busy workers' deques (work stealing). Powers parallel streams.
- **Virtual threads (Java 21)**: very lightweight threads scheduled by the JVM onto a few carrier threads. Ideal for blocking I/O workloads (thread-per-request at massive scale). Create with `Executors.newVirtualThreadPerTaskExecutor()`; don't pool them; they don't speed up CPU-bound work. Since

Java 24, `synchronized` no longer pins them to carriers. Structured concurrency (preview) treats related tasks as one unit.

CODE · COMPLETABLEFUTURE AND VIRTUAL THREADS

```
CompletableFuture<User> user = CompletableFuture.supplyAsync(() -> userApi.get(id), pool);
CompletableFuture<List<Order>> orders = CompletableFuture.supplyAsync(() -> orderApi.list(id), pool);

Profile profile = user.thenCombine(orders, Profile::new)
        .orTimeout(2, TimeUnit.SECONDS)
        .exceptionally(ex -> Profile.empty())
        .join();

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    urls.forEach(u -> executor.submit(() -> download(u)));
}   // waits for all tasks to finish
```

INTERVIEW CHECK · can you answer these out loud?

Q1 sleep() vs wait()? volatile vs synchronized?
Q2 How do you detect and prevent deadlocks?
Q3 Explain how ConcurrentHashMap achieves thread safety.
Q4 Virtual threads vs platform threads: when would you use each?
Q5 Write a producer-consumer solution using BlockingQueue.

TOPIC CHECKLIST · 39 ITEMS

☐ Process vs thread	☐ Thread lifecycle	☐ Creating threads
☐ Extending Thread	☐ Implementing Runnable	☐ Callable
☐ Future	☐ ExecutorService	☐ Thread pools
☐ ScheduledExecutorService	☐ Thread priorities	☐ Daemon threads
☐ Synchronization	☐ Intrinsic locks and monitors	☐ synchronized methods and blocks
☐ volatile	☐ Atomic classes	☐ Race conditions
☐ Deadlocks	☐ Starvation	☐ Livelock
☐ Thread safety	☐ wait(), notify(), notifyAll()	☐ sleep() and join()
☐ Lock and ReentrantLock	☐ ReadWriteLock	☐ Semaphores
☐ Latches	☐ Barriers	☐ BlockingQueue
☐ Concurrent collections	☐ Java Memory Model	☐ Happens-before
☐ Visibility and atomicity	☐ CompletableFuture	☐ Fork/Join framework
☐ Work stealing	☐ Virtual threads	☐ Structured concurrency

19 JVM Internals

KEY REVISION NOTES

- **Class loaders**: Bootstrap (core JDK classes), Platform (other JDK modules) and Application/System (your classpath). They follow the parent-delegation model: a loader asks its parent first, which prevents core classes from being replaced.
- **Class lifecycle**: Loading (read bytecode) → Linking (verification of bytecode safety, preparation of static fields with defaults, resolution of symbolic references) → Initialization (run static initializers, triggered on first active use).
- **Runtime data areas**: Heap (all objects and arrays, shared by threads, managed by GC); Stack (one per thread, a frame per method call with local variables and operand stack); PC register (current instruction per thread); Native method stack; Method area / Metaspace (class metadata, in native memory, which replaced PermGen in Java 8).
- **GC basics**: objects reachable from GC roots (local variables in active frames, static fields, active threads, JNI references) are alive; everything else can be collected. Reference cycles are not a problem.

- **Generational GC**: most objects die young. New objects go to Eden; survivors move between survivor spaces and are promoted to the old generation after surviving several minor GCs. Old-gen collections are more expensive.
- **Collectors**: Serial (single thread, small heaps), Parallel (throughput), G1 (default since Java 9, region-based, pause-time goals), ZGC (sub-millisecond pauses, generational since Java 21), Shenandoah (low pause, concurrent compaction).
- **JIT**: the interpreter runs bytecode first; hot methods are compiled to native code by C1 (fast) and then C2 (heavily optimised) in tiered compilation. Optimisations include inlining, loop unrolling and escape analysis (allocating non-escaping objects on the stack or eliminating them via scalar replacement).
- **Memory leaks in Java**: objects still referenced but no longer needed, e.g. growing static collections, caches without eviction, unclosed resources, listeners never removed, `ThreadLocal` values in pooled threads, inner classes holding outer references.
- **Errors**: `OutOfMemoryError: Java heap space` (heap full, possibly a leak), `Metaspace` (too many classes, classloader leaks), `StackOverflowError` (deep or infinite recursion).
- **Tools and flags**: `-Xms`/`-Xmx` (heap size), `-Xss` (thread stack size), `-XX:+HeapDumpOnOutOfMemoryError`, `-Xlog:gc*`. `jcmd`, `jstack` (thread dumps for deadlocks and hangs), `jmap` (heap dumps), Eclipse MAT or VisualVM for analysis, Java Flight Recorder (low-overhead profiling) and Java Mission Control to view recordings.

CODE · PRODUCTION DIAGNOSIS TOOLKIT

```
java -Xms512m -Xmx2g -XX:+UseG1GC \
     -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp \
     -Xlog:gc*:file=gc.log -jar app.jar

jcmd <pid> Thread.print                  # thread dump (deadlocks, hangs)
jcmd <pid> GC.heap_dump /tmp/heap.hprof  # heap dump -> analyse in MAT
jcmd <pid> JFR.start duration=60s filename=rec.jfr
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Explain stack vs heap memory.
Q2 How does the GC decide which objects to collect?
Q3 How would you diagnose high CPU or an OutOfMemoryError in production?
Q4 PermGen vs Metaspace? What is parent delegation?

TOPIC CHECKLIST · 29 ITEMS

☐ JVM architecture	☐ Class loader subsystem	☐ Loading, linking, initialization
☐ Bytecode verification	☐ Runtime data areas	☐ Heap
☐ Stack	☐ Program counter register	☐ Native method stack
☐ Method area and metaspace	☐ Garbage collection	☐ GC roots
☐ Generational GC	☐ Common garbage collectors	☐ G1 GC
☐ ZGC	☐ Shenandoah GC	☐ Just-in-time compilation
☐ Interpreter vs JIT	☐ Escape analysis	☐ Object allocation
☐ Memory leaks	☐ OutOfMemoryError	☐ StackOverflowError
☐ JVM options	☐ Heap dumps	☐ Thread dumps
☐ Java Flight Recorder	☐ Java Mission Control	

20 Reflection and Annotations

KEY REVISION NOTES

- **Class object**: get it via `obj.getClass()`, `MyClass.class` or `Class.forName("com.x.MyClass")`. From it, inspect fields, methods, constructors, annotations and modifiers.
- **getMethods vs getDeclaredMethods**: `getMethods()` returns public methods including inherited ones; `getDeclaredMethods()` returns all methods declared in the class itself, of any visibility. `setAccessible(true)` bypasses access checks, but modules can block it unless the package is `opens`.
- **Dynamic use**: `clazz.getDeclaredConstructor().newInstance()`, `method.invoke(target, args)`, `field.set(obj, value)`. This is how Spring creates beans, Jackson maps JSON, Hibernate populates entities and JUnit finds tests.
- **Annotations** are metadata. Retention: `SOURCE` (discarded by compiler, e.g. `@Override`), `CLASS` (in bytecode but not visible at runtime, the default), `RUNTIME` (readable via reflection, which frameworks need). Targets: `TYPE`, `METHOD`, `FIELD`, `PARAMETER`, etc.
- **Built-in and meta-annotations**: `@Override`, `@Deprecated`, `@SuppressWarnings`, `@FunctionalInterface`, `@SafeVarargs`. Meta-annotations: `@Retention`, `@Target`, `@Documented`, `@Inherited`, `@Repeatable`.
- **Annotation processing**: compile-time code generation through the `javax.annotation.processing` API (Lombok, MapStruct, Dagger), with no runtime reflection cost.
- **Dynamic proxies**: `java.lang.reflect.Proxy` creates proxy objects for interfaces at runtime, routing calls through an `InvocationHandler`. Class-based proxies use bytecode libraries (CGLIB, ByteBuddy). Spring AOP, `@Transactional` and Hibernate lazy loading rely on proxies.
- **Costs**: slower than direct calls, breaks encapsulation, loses compile-time checks, and can be restricted by modules and security policies.

CODE · CUSTOM RUNTIME ANNOTATION

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Audit { String action(); }

for (Method m : service.getClass().getDeclaredMethods()) {
    Audit a = m.getAnnotation(Audit.class);
    if (a != null) System.out.println(m.getName() + " -> " + a.action());
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 How does Spring discover classes annotated with `@Component`?
Q2 What is the difference between JDK dynamic proxies and CGLIB proxies?
Q3 Why must framework annotations use RUNTIME retention?

TOPIC CHECKLIST · 15 ITEMS

☐ Reflection API	☐ Class object	☐ Inspecting methods and fields
☐ Creating objects dynamically	☐ Invoking methods dynamically	☐ Runtime annotations
☐ Built-in annotations	☐ Custom annotations	☐ Retention policies
☐ Annotation targets	☐ Repeatable annotations	☐ Meta-annotations
☐ Annotation processing	☐ Dynamic proxies	☐ Reflection security and performance

21 JDBC

KEY REVISION NOTES

- **Architecture**: your code uses the JDBC API (`java.sql`); a vendor-specific driver (PostgreSQL, MySQL) translates calls to the database protocol. Modern drivers (Type 4) are pure Java and auto-registered.
- **Typical flow**: obtain a `Connection` (from a `DataSource`) → create a `PreparedStatement` → bind parameters → execute → iterate the `ResultSet` → close everything with try-with-resources.
- **Statement vs PreparedStatement**: `PreparedStatement` is precompiled and binds parameters safely with `?`, which prevents SQL injection and allows plan reuse. Never concatenate user input into SQL.
- **Execute methods**: `executeQuery()` for SELECT (returns `ResultSet`), `executeUpdate()` for INSERT/UPDATE/DELETE (returns affected row count), `execute()` for anything.
- **Transactions**: `conn.setAutoCommit(false)`, then `commit()` on success or `rollback()` in the catch block. Savepoints allow partial rollback.
- **Batch processing**: `addBatch()` and `executeBatch()` send many statements in one round trip; much faster for bulk inserts.
- **CallableStatement** calls stored procedures: `{call calc_bonus(?, ?)}` with registered OUT parameters.
- **Connection pooling**: opening a DB connection is expensive (network, auth). Pools such as HikariCP (Spring Boot's default) keep connections open and reuse them. `DataSource` is preferred over `DriverManager` because it supports pooling and configuration.
- **Errors**: `SQLException` provides `getSQLState()` and `getErrorCode()`. Spring translates them into its unchecked `DataAccessException` hierarchy.

CODE · SAFE QUERY WITH PREPAREDSTATEMENT

```
String sql = "SELECT id, name FROM users WHERE email = ? AND active = ?";
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, email);
    ps.setBoolean(2, true);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            users.add(new User(rs.getLong("id"), rs.getString("name")));
        }
    }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Statement vs PreparedStatement? How does it prevent SQL injection?
Q2 Why is connection pooling important?
Q3 How do you manage a transaction in plain JDBC?

TOPIC CHECKLIST · 19 ITEMS

☐ JDBC architecture	☐ JDBC drivers	☐ Database connectivity
☐ DriverManager	☐ Connection	☐ Statement
☐ PreparedStatement	☐ CallableStatement	☐ ResultSet
☐ CRUD operations	☐ SQL parameter binding	☐ Transactions
☐ Commit and rollback	☐ Batch processing	☐ Stored procedures
☐ Connection pooling	☐ JDBC exception handling	☐ SQL injection prevention
☐ DataSource		

22 SQL and Database Fundamentals

KEY REVISION NOTES

- **Keys and constraints**: primary key (unique, not null), foreign key (referential integrity), `UNIQUE`, `NOT NULL`, `CHECK`, `DEFAULT`.
- **Normalization**: 1NF (atomic values, no repeating groups), 2NF (no partial dependency on part of a composite key), 3NF (no transitive dependency on non-key columns). Denormalize deliberately for read performance.
- **Joins**: `INNER` (matching rows only), `LEFT` (all left rows + matches, nulls otherwise), `RIGHT`, `FULL OUTER`, `CROSS` (cartesian product), self join (e.g. employee and manager in one table).
- **Logical execution order**: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET. That's why a SELECT alias cannot be used in WHERE.
- **WHERE vs HAVING**: WHERE filters rows before grouping; HAVING filters groups after aggregation.
- **CTEs and window functions**: `WITH` makes queries readable and supports recursion. Window functions (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`, `SUM(...) OVER (PARTITION BY ... ORDER BY ...)`) compute across related rows without collapsing them.
- **Indexes**: usually B-trees. They speed up lookups, joins and sorting but slow down writes and use storage. Composite indexes follow the leftmost-prefix rule. Wrapping a column in a function, leading wildcards (`LIKE '%x'`) or implicit type casts can prevent index use. Index foreign keys and frequently filtered columns.
- **ACID**: Atomicity (all or nothing), Consistency (constraints hold), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). *MySQL InnoDB largely prevents phantoms at REPEATABLE READ through next-key locking.
- **Locks and deadlocks**: row and table locks, `SELECT ... FOR UPDATE`. Databases detect deadlocks and abort one transaction; keep transactions short and access tables in a consistent order.
- **Optimisation**: use `EXPLAIN` / `EXPLAIN ANALYZE` to read execution plans (sequential scan vs index scan, join types). Select only needed columns, paginate, avoid N+1 queries, use keyset pagination for deep pages.
- **Others**: views (saved queries), stored procedures, and database migrations (Flyway/Liquibase) for versioned schema changes. Know both MySQL and PostgreSQL basics.

Isolation level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED (PostgreSQL default)	Prevented	Possible	Possible
REPEATABLE READ (MySQL InnoDB default)	Prevented	Prevented	Possible*
SERIALIZABLE	Prevented	Prevented	Prevented

CODE · TOP-2 SALARIES PER DEPARTMENT (WINDOW FUNCTION)

```
WITH ranked AS (
    SELECT name, dept_id, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT d.name AS dept, r.name, r.salary
FROM ranked r JOIN departments d ON d.id = r.dept_id
WHERE r.rnk <= 2;

-- Departments with more than 5 employees
SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id HAVING COUNT(*) > 5;
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Find the second highest salary (several ways).
Q2 WHERE vs HAVING? RANK vs DENSE_RANK vs ROW_NUMBER?
Q3 How do indexes work and when can they hurt?
Q4 Explain isolation levels and the anomalies each prevents.

TOPIC CHECKLIST · 24 ITEMS

☐ Relational concepts	☐ Database design	☐ Primary and foreign keys
☐ Constraints	☐ Normalization	☐ SELECT queries
☐ WHERE and ORDER BY	☐ GROUP BY and HAVING	☐ Joins
☐ Subqueries	☐ Common table expressions	☐ Window functions
☐ Indexes	☐ Transactions	☐ ACID properties
☐ Isolation levels	☐ Locks	☐ Deadlocks
☐ Query optimization	☐ Execution plans	☐ Stored procedures
☐ Views	☐ Database migrations	☐ MySQL and PostgreSQL

23 Build Tools and Dependency Management

KEY REVISION NOTES

- **Maven coordinates**: `groupId:artifactId:version` identify every artifact. `SNAPSHOT` versions are mutable development builds; release versions are immutable.
- **Lifecycle**: validate → compile → test → package → verify → install (to local `~/.m2`) → deploy (to a remote repository). Running a phase runs all earlier phases. `mvn clean` is a separate lifecycle.
- **Scopes**: `compile` (default, everywhere), `provided` (needed to compile but supplied at runtime, e.g. servlet API), `runtime` (e.g. JDBC driver), `test` (JUnit, Mockito), `system` (avoid), `import` (import a BOM into `dependencyManagement`).
- **Transitive dependencies and conflicts**: Maven uses "nearest wins" mediation. Inspect with `mvn dependency:tree`, fix with `<exclusions>` or by pinning versions in `<dependencyManagement>`. BOMs (like `spring-boot-dependencies`) align versions across many libraries.
- **Plugins** do the work in each phase (compiler, surefire for unit tests, failsafe for integration tests, jar, spring-boot plugin). **Profiles** switch configuration per environment.
- **Multi-module projects**: a parent POM with `<modules>` shares configuration and builds modules in dependency order. Packaging can be `jar`, `war` or `pom`.
- **Gradle**: build scripts in Groovy or Kotlin DSL, a task graph, incremental builds and a build cache (often faster). Configurations: `implementation` (not leaked to consumers), `api` (exposed), `compileOnly`, `runtimeOnly`, `testImplementation`.
- **Wrappers**: `mvnw` / `gradlew` pin the tool version so every developer and CI server builds the same way (reproducible builds).
- **Semantic versioning**: MAJOR (breaking changes).MINOR (new compatible features).PATCH (bug fixes). Artifact repositories: Maven Central, Nexus, Artifactory, GitHub Packages.

CODE · USEFUL MAVEN COMMANDS

```
mvn clean verify                    # full build with tests
mvn -DskipTests package             # build JAR without running tests
mvn dependency:tree -Dincludes=com.fasterxml.jackson.core
mvn -pl orders-service -am install   # build one module + its dependencies
./mvnw spring-boot:run
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What does the `provided` scope mean? Give an example.
Q2 How do you resolve a dependency version conflict?
Q3 `mvn install` vs `mvn deploy`? Gradle `api` vs `implementation`?

TOPIC CHECKLIST · 21 ITEMS

☐ Maven lifecycle	☐ pom.xml	☐ Dependencies
☐ Dependency scopes	☐ Transitive dependencies	☐ Dependency exclusions
☐ Plugins	☐ Profiles	☐ Multi-module projects
☐ Packaging JAR/WAR	☐ Maven wrapper	☐ Gradle build scripts
☐ Gradle plugins	☐ Dependency configurations	☐ Gradle tasks
☐ Multi-project builds	☐ Gradle wrapper	☐ Semantic versioning
☐ Dependency conflicts	☐ Reproducible builds	☐ Artifact repositories

24 Git and Development Tools

KEY REVISION NOTES

- **Three areas**: working directory → staging area (`git add`) → repository (`git commit`). Write small, focused commits with clear messages.
- **Branching workflow**: create a feature branch, push it, open a pull request, get a code review, pass CI, merge. Keep `main` deployable.
- **merge vs rebase**: `merge` preserves history and adds a merge commit. `rebase` replays your commits on top of another branch for a linear history but rewrites commit IDs. Never rebase branches others are working on.
- **reset vs revert**: `reset` moves the branch pointer (rewrites history: `--soft` keeps changes staged, `--mixed` unstaged, `--hard` discards). `revert` creates a new commit that undoes a previous one and is safe for shared branches.
- **Other commands**: `cherry-pick` applies a specific commit to the current branch; `stash` shelves uncommitted work; `fetch` downloads without merging while `pull` = fetch + merge (or rebase); `log --oneline --graph`; `bisect` finds the commit that introduced a bug.
- **Conflict resolution**: edit the conflicting sections between markers, test, `git add`, then continue the merge or rebase.
- **.gitignore**: exclude `target/`, `build/`, `.idea/`, `*.iml`, `.env` and anything with secrets. If a secret is ever committed, rotate it; deleting it later doesn't remove it from history.
- **IDEs**: IntelliJ IDEA (industry standard for Java), Eclipse, VS Code with Java extensions. Learn refactoring shortcuts and navigation.
- **Debugging**: breakpoints, step over/into/out, evaluate expressions, watches, conditional breakpoints (stop only when `id == 42`), exception breakpoints, and remote debugging with `-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005`.
- **Static analysis**: Checkstyle (code style), SpotBugs (bug patterns), PMD, SonarQube (quality gates, code smells, coverage, security hotspots) integrated into CI.

CODE · EVERYDAY GIT

```
git switch -c feature/order-api
git add -p                        # stage selectively
git commit -m "Add order creation endpoint"
git fetch origin && git rebase origin/main
git push -u origin feature/order-api
git revert <sha>                  # safely undo a pushed commit
```

INTERVIEW CHECK · can you answer these out loud?

Q1 merge vs rebase? When would you use each?
Q2 How do you undo a commit that has already been pushed?
Q3 git fetch vs git pull?

TOPIC CHECKLIST · 22 ITEMS

☐ Git basics	☐ Repository initialization	☐ Commit and branch
☐ Merge and rebase	☐ Cherry-pick	☐ Conflict resolution
☐ Reset and revert	☐ Pull requests	☐ Code reviews
☐ .gitignore	☐ GitHub	☐ IntelliJ IDEA
☐ Eclipse	☐ VS Code	☐ Debugging
☐ Breakpoints	☐ Conditional breakpoints	☐ Remote debugging
☐ Static analysis	☐ Checkstyle	☐ SpotBugs
☐ SonarQube		

PART B

Java Backend Development

Web fundamentals, Spring, Spring Boot, REST, JPA, security, transactions, testing, microservices, caching, messaging, observability, DevOps and system design (topics 25-39).

25 Web Fundamentals

KEY REVISION NOTES

- **Client-server and HTTP**: HTTP is a stateless request/response protocol. A request has a method, URL, headers and optional body; a response has a status code, headers and body.
- **Methods**: `GET` reads (safe and idempotent), `POST` creates or triggers actions (not idempotent), `PUT` replaces a resource (idempotent), `PATCH` partially updates, `DELETE` removes (idempotent). Idempotent means repeating the request has the same effect as doing it once.
- **401 vs 403**: 401 means not authenticated (who are you?); 403 means authenticated but not allowed.
- **REST**: resources identified by URLs using nouns and plurals (`/api/users/42/orders`), standard HTTP methods, stateless requests, cacheable responses, and representations such as JSON. Avoid verbs in URLs (`/getUser`).
- **JSON vs XML**: JSON is lighter and the default for modern APIs; XML still appears in SOAP and legacy integrations.
- **State**: cookies are stored by the browser and sent automatically. A server-side session is identified by a cookie (`JSESSIONID`). Token-based auth (JWT in the `Authorization: Bearer` header) keeps the server stateless.
- **HTTPS and TLS**: HTTP over TLS provides encryption, integrity and server authentication through certificates issued by a certificate authority.
- **CORS**: the browser blocks cross-origin requests unless the server returns `Access-Control-Allow-Origin` headers. Non-simple requests trigger a preflight `OPTIONS` request. CORS is enforced by browsers, not by servers or Postman.
- **What happens when you type a URL**: DNS resolves the domain to an IP → TCP connection → TLS handshake → HTTP request → server processes → response → browser renders and fetches more resources.

Range	Meaning	Codes to know
2xx	Success	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405, 409 Conflict, 415, 422, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

INTERVIEW CHECK · can you answer these out loud?

Q1 PUT vs PATCH vs POST? Which methods are idempotent?
Q2 401 vs 403?
Q3 What happens when you type a URL into the browser?
Q4 What is CORS and why does it exist?

TOPIC CHECKLIST · 13 ITEMS

☐ Client-server architecture	☐ HTTP and HTTPS	☐ Request and response
☐ HTTP methods	☐ HTTP status codes	☐ Headers and cookies
☐ Sessions	☐ REST architecture	☐ JSON and XML
☐ CORS	☐ DNS basics	☐ TLS basics
☐ Authentication vs authorization		

26 Servlets and Jakarta EE

KEY REVISION NOTES

- **Servlet lifecycle**: the container loads the class and calls `init()` once, then `service()` for every request (dispatching to `doGet`, `doPost`, etc.), and `destroy()` once at shutdown.
- **Thread safety**: a container (Tomcat, Jetty, Undertow) usually creates one servlet instance and handles each request on a separate thread. Keep servlets stateless; never store request data in instance fields.
- **Filters** intercept requests and responses in a chain (logging, authentication, compression, CORS). Spring Security is built on a servlet filter. **Listeners** react to lifecycle events (context started, session created).
- **Session management**: `HttpSession` stored server-side, identified by a `JSESSIONID` cookie (or URL rewriting). Sessions don't scale horizontally without sticky sessions or a shared session store (e.g. Spring Session with Redis).
- **forward vs redirect**: forward happens on the server (same request, URL unchanged); redirect sends a 3xx status so the browser makes a new request (URL changes). Post/Redirect/Get prevents duplicate form submissions.
- **JSP and JSTL**: JSPs are compiled into servlets; JSTL provides tags for loops and conditions. Largely legacy now, replaced by Thymeleaf or a separate frontend calling REST APIs.
- **MVC and WAR**: Model-View-Controller separates data, presentation and request handling. A WAR file is deployed into an external server; Spring Boot usually embeds the server in an executable JAR instead.
- **Jakarta EE**: Java EE moved to the Eclipse Foundation, and from Jakarta EE 9 the packages changed from `javax.*` to `jakarta.*`. Spring Boot 3+ uses `jakarta.*`. Spring MVC's `DispatcherServlet` is itself a servlet.

INTERVIEW CHECK · can you answer these out loud?

Q1 Explain the servlet lifecycle. Are servlets thread-safe?
Q2 Forward vs redirect?
Q3 What is a filter and how does it differ from a Spring interceptor?

TOPIC CHECKLIST · 14 ITEMS

☐ Servlet lifecycle	☐ Servlet container	☐ Request and response objects
☐ Servlet configuration	☐ Filters	☐ Listeners
☐ Session management	☐ Cookies	☐ JSP fundamentals
☐ JSTL	☐ MVC architecture	☐ WAR packaging
☐ Tomcat	☐ Jakarta EE overview	

27 Spring Framework

KEY REVISION NOTES

- **IoC and DI**: with Inversion of Control, the Spring container creates objects (beans) and manages their lifecycle. Dependency Injection is how the container supplies a bean's dependencies instead of the bean creating them. Result: loose coupling, easy testing with mocks, swappable implementations.
- **Constructor injection** is preferred: dependencies can be `final`, required dependencies are explicit, objects are always fully initialised, and tests can call the constructor directly. With a single constructor, `@Autowired` is optional. Field injection hides dependencies and hurts testability.
- **ApplicationContext vs BeanFactory**: `ApplicationContext` extends `BeanFactory` and adds eager singleton creation, events, internationalisation, environment and profile support. Always use ApplicationContext in practice.
- **Bean lifecycle**: instantiate → inject dependencies → Aware callbacks → `BeanPostProcessor` before-init → `@PostConstruct` / `afterPropertiesSet` / init-method → `BeanPostProcessor` after-init (proxies for AOP and transactions are created here) → in use → `@PreDestroy` on shutdown.
- **Scopes**: `singleton` (default, one per container), `prototype` (new instance each time it's requested), and web scopes `request`, `session`, `application`, `websocket`. Injecting a prototype into a singleton gives only one instance; use `ObjectProvider<T>` or a lookup method.
- **Singleton thread safety**: Spring singletons are shared across all request threads, so keep them stateless (no mutable instance fields).
- **Stereotypes**: `@Component` (generic), `@Service` (business logic), `@Repository` (data access, with exception translation to `DataAccessException`), `@Controller` (web). Discovered via `@ComponentScan`.
- **@Bean vs @Component**: `@Component` annotates your own classes; `@Bean` methods inside `@Configuration` classes create beans from third-party classes or need custom construction logic. `@Configuration` classes are proxied so calls between `@Bean` methods return the same singleton.
- **Multiple candidates**: when two beans implement the same interface, resolve with `@Primary` or `@Qualifier("name")`, or inject `List<Interface>` to get all of them.
- **Configuration**: `@Value("${app.timeout}")` for single values, profiles (`@Profile("dev")`), properties or YAML files. SpEL (`#{...}`) evaluates expressions.
- **AOP**: separates cross-cutting concerns (logging, transactions, security, metrics). Terms: aspect, advice (`@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`), pointcut (where), join point (a method execution). Spring AOP is proxy-based, so only external calls through the proxy are intercepted, and only public methods by default.
- **Events**: publish with `ApplicationEventPublisher` and listen with `@EventListener` or `@TransactionalEventListener` (e.g. send email only after commit). Decouples components.

CODE · CONSTRUCTOR INJECTION AND AN ASPECT

```
@Service
public class OrderService {
    private final OrderRepository repo;
    private final PaymentClient payments;

    public OrderService(OrderRepository repo, PaymentClient payments) {
        this.repo = repo;            // no @Autowired needed
        this.payments = payments;
    }
}

@Aspect @Component
class TimingAspect {
    @Around("@annotation(com.shop.Timed)")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long t = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature(), (System.nanoTime() - t) / 1_000_000); }
    }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is IoC and DI? Why is constructor injection preferred?
Q2 Are Spring singleton beans thread-safe?
Q3 @Component vs @Bean? How do you resolve two beans of the same type?
Q4 How does Spring AOP work internally and what are its limitations?

TOPIC CHECKLIST · 25 ITEMS

☐ Spring architecture	☐ Inversion of Control	☐ Dependency Injection
☐ Bean lifecycle	☐ Bean scopes	☐ ApplicationContext
☐ Configuration classes	☐ Component scanning	☐ @Component
☐ @Service	☐ @Repository	☐ @Controller
☐ @Autowired	☐ Constructor injection	☐ @Bean
☐ @Configuration	☐ @Value	☐ Profiles
☐ Properties and YAML	☐ Spring Expression Language	☐ AOP
☐ Proxies	☐ Cross-cutting concerns	☐ Spring events
☐ Application events		

28 Spring Boot

KEY REVISION NOTES

- **What Boot adds**: opinionated defaults on top of Spring: auto-configuration, starter dependencies, an embedded server (Tomcat by default), externalized configuration, Actuator, and an executable fat JAR.
- **@SpringBootApplication** = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan` (scans the package of the main class and its sub-packages).
- **Auto-configuration**: Boot reads candidate configuration classes (listed in `META-INF/spring/...AutoConfiguration.imports` in Boot 3) and applies them conditionally: `@ConditionalOnClass` (library on classpath), `@ConditionalOnMissingBean` (you haven't defined your own), `@ConditionalOnProperty`. Your own beans win. Run with `--debug` to see the condition evaluation report; disable with `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.
- **Starters**: curated dependency bundles such as `spring-boot-starter-web`, `-data-jpa`, `-security`, `-validation`, `-test`, `-actuator`, with versions managed by the Boot BOM.
- **Externalized configuration**: properties can come from many sources. Simplified precedence (highest first): command-line arguments → environment variables → profile-specific files (`application-prod.yml`) → `application.yml`. Activate profiles with `spring.profiles.active=prod`.

- **@ConfigurationProperties**: binds a group of properties to a typed class or record with validation; better than many scattered `@Value` annotations.
- **Actuator**: production endpoints such as `/actuator/health`, `/info`, `/metrics`, `/prometheus`, `/loggers`, with liveness and readiness health groups. Expose only what you need and secure them.
- **Logging**: SLF4J API with Logback by default. Configure levels with `logging.level.com.shop=DEBUG`. Use parameterised messages: `log.info("Order {} created", id)`.
- **Error handling and startup**: default `/error` handling, customised via `@RestControllerAdvice`. `CommandLineRunner` / `ApplicationRunner` run code at startup.
- **Packaging and deployment**: `mvn package` produces an executable JAR run with `java -jar`. Boot 3+ requires Java 17+ and uses Jakarta EE. Supports layered JARs, buildpacks (`spring-boot:build-image`) and GraalVM native images.

CODE · TYPED CONFIGURATION

```
# application.yml
app:
  payment:
    base-url: https://pay.example.com
    timeout: 3s

@ConfigurationProperties(prefix = "app.payment")
@Validated
public record PaymentProperties(@NotBlank String baseUrl, Duration timeout) {}

// enable with @EnableConfigurationProperties(PaymentProperties.class)
// or @ConfigurationPropertiesScan on the main class
```

INTERVIEW CHECK · can you answer these out loud?

Q1 How does Spring Boot auto-configuration work?
Q2 How do you override or disable an auto-configured bean?
Q3 What does @SpringBootApplication do?
Q4 How does Boot decide which property value wins?

TOPIC CHECKLIST · 18 ITEMS

☐ Spring Boot architecture	☐ Auto-configuration	☐ Starter dependencies
☐ Embedded servers	☐ Project initialization	☐ Configuration properties
☐ Profiles	☐ Externalized configuration	☐ application.properties
☐ application.yml	☐ Spring Boot CLI	☐ Actuator
☐ Health checks	☐ Metrics	☐ Logging
☐ Custom error handling	☐ Application startup	☐ Packaging and deployment

29 Spring MVC and REST APIs

KEY REVISION NOTES

- **Request flow**: client → `DispatcherServlet` (front controller) → `HandlerMapping` finds the controller method → `HandlerAdapter` invokes it (argument resolvers bind path variables, params and body) → return value → `HttpMessageConverter` (Jackson) writes JSON → response.
- **@RestController** = `@Controller` + `@ResponseBody`, so return values are serialised to the response body instead of resolved as view names.
- **Binding**: `@PathVariable` identifies a resource (`/users/{id}`); `@RequestParam` handles filters and options (`?status=PAID`); `@RequestBody` deserialises JSON into an object; `@RequestHeader` reads headers.
- **ResponseEntity**: control status and headers, e.g. `ResponseEntity.created(location).body(dto)` for 201 or `ResponseEntity.noContent().build()` for 204.

- **DTOs**: never expose JPA entities directly. DTOs prevent lazy-loading exceptions, infinite JSON recursion, over-posting (clients setting fields they shouldn't), and coupling your API to your schema. Map with MapStruct or by hand; records are ideal DTOs.
- **Validation**: annotate DTO fields with `@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Positive`, `@Pattern`, and the parameter with `@Valid`. Failures throw `MethodArgumentNotValidException`, which you map to a 400 response. Custom validators implement `ConstraintValidator`.
- **Global exception handling**: `@RestControllerAdvice` with `@ExceptionHandler` methods maps exceptions to consistent error responses. Spring 6 supports `ProblemDetail` (RFC 9457) as a standard error format.
- **Pagination and sorting**: accept a `Pageable` (`?page=0&size=20&sort=createdAt,desc`) and return a `Page` or a slimmer DTO. Always cap page size.
- **Content negotiation**: the `Accept` and `Content-Type` headers decide formats (`produces`/`consumes`). File uploads use `MultipartFile`; downloads return a `Resource` with `Content-Disposition`.
- **API design**: versioning (`/api/v1`, header, or media type), consistent naming, proper status codes, idempotency keys for unsafe operations such as payments, and rate limiting (429) at the gateway or with Bucket4j.
- **Documentation**: springdoc-openapi generates an OpenAPI spec and Swagger UI from your controllers.

CODE · CONTROLLER + GLOBAL ERROR HANDLER

```
@RestController
@RequestMapping("/api/v1/orders")
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }

    @PostMapping
    ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrderRequest req) {
        OrderDto dto = service.create(req);
        return ResponseEntity.created(URI.create("/api/v1/orders/" + dto.id())).body(dto);
    }

    @GetMapping
    Page<OrderDto> list(@RequestParam(required = false) Status status, Pageable pageable) {
        return service.find(status, pageable);
    }
}

@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage());
    }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Explain the request flow through DispatcherServlet.
Q2 @Controller vs @RestController? @PathVariable vs @RequestParam?
Q3 Why use DTOs instead of entities in APIs?
Q4 How do you handle validation errors globally?

TOPIC CHECKLIST · 25 ITEMS

☐ MVC architecture	☐ DispatcherServlet	☐ Controllers
☐ @RestController	☐ Request mappings	☐ Path variables
☐ Request parameters	☐ Request bodies	☐ Response entities
☐ DTOs	☐ Request/response validation	☐ Bean Validation
☐ @Valid	☐ Global exception handling	☐ @ControllerAdvice
☐ Content negotiation	☐ Pagination	☐ Sorting
☐ Filtering	☐ File uploads and downloads	☐ API versioning
☐ OpenAPI and Swagger	☐ REST API design	☐ Idempotency
☐ Rate limiting concepts		

30 Spring Data JPA and Hibernate

KEY REVISION NOTES

- **Layers**: JPA is the specification (`jakarta.persistence`), Hibernate is the most common implementation, and Spring Data JPA adds repository interfaces on top so you write little or no boilerplate.
- **Entity states**: transient (new, not tracked), managed (tracked by the persistence context), detached (was managed, context closed), removed (scheduled for deletion).
- **Persistence context**: a first-level cache scoped to the transaction. Loading the same ID twice returns the same object. **Dirty checking** compares managed entities with their snapshots at flush time and issues UPDATEs automatically, so you don't need to call `save()` on a managed entity.
- **ID generation**: `IDENTITY` (auto-increment; disables JDBC insert batching), `SEQUENCE` (preferred for PostgreSQL, supports batching), `UUID`. Avoid `AUTO` surprises by choosing explicitly.
- **Relationships**: the owning side holds the foreign key (usually `@ManyToOne`); the inverse side declares `mappedBy`. Keep both sides in sync with helper methods. Prefer `@ManyToOne` over unidirectional `@OneToMany`.
- **Fetch types**: defaults are EAGER for `@ManyToOne`/`@OneToOne` and LAZY for `@OneToMany`/`@ManyToMany`. Make everything LAZY and fetch what you need per use case. Accessing a lazy association outside a transaction throws `LazyInitializationException`; don't rely on Open Session in View.
- **N+1 problem**: one query loads N parents, then N more queries load each parent's children. Detect it by enabling SQL logging or Hibernate statistics. Fix with `JOIN FETCH`, `@EntityGraph`, batch fetching (`hibernate.default_batch_fetch_size`), or DTO projections.
- **Cascading**: `CascadeType.PERSIST`/`MERGE`/`REMOVE`/`ALL` propagate operations from parent to children; `orphanRemoval = true` deletes children removed from a collection. Don't cascade REMOVE on `@ManyToMany`.
- **Queries**: derived queries from method names (`findByStatusAndCreatedAtAfter`), `@Query` with JPQL (entity names, not tables) or `nativeQuery = true`, Specifications and the Criteria API for dynamic filters, interface or record projections for read-only views.
- **Locking**: optimistic locking with a `@Version` column throws `OptimisticLockException` if someone else updated first (good for low contention). Pessimistic locking (`@Lock(PESSIMISTIC_WRITE)`, i.e. `SELECT ... FOR UPDATE`) blocks others (high contention, e.g. inventory).
- **Second-level cache**: shared across sessions (Ehcache, Caffeine via JCache) for rarely changing reference data.
- **Auditing**: `@EnableJpaAuditing` with `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`.
- **Entity pitfalls**: avoid Lombok `@Data`/`@ToString` on entities (triggers lazy loading and recursion); implement `equals`/`hashCode` carefully (business key or ID with null checks); set `spring.jpa.hibernate.ddl-auto=validate` in production and manage schemas with Flyway or Liquibase.

CODE · FIXING N+1

```
// Problem: 1 query for orders + 1 query per order for its items
List<Order> orders = orderRepo.findAll();
orders.forEach(o -> o.getItems().size());

// Fix 1: fetch join
@Query("select distinct o from Order o join fetch o.items where o.status = :status")
List<Order> findWithItems(@Param("status") Status status);

// Fix 2: entity graph
@EntityGraph(attributePaths = {"items", "customer"})
List<Order> findByStatus(Status status);

// Optimistic locking
@Entity class Product { @Id Long id; @Version Long version; int stock; }
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is the N+1 problem and how do you fix it?
Q2 Explain the persistence context and dirty checking.
Q3 Optimistic vs pessimistic locking?
Q4 `findById` vs `getReferenceById`? `save` vs `saveAndFlush`?

TOPIC CHECKLIST · 35 ITEMS

☐ ORM concepts	☐ JPA specification	☐ Hibernate
☐ Entities	☐ Entity lifecycle	☐ @Entity
☐ Primary key generation	☐ @Column	☐ Entity relationships
☐ One-to-one	☐ One-to-many	☐ Many-to-one
☐ Many-to-many	☐ Cascading	☐ Fetch types
☐ Lazy vs eager loading	☐ JPQL	☐ Native SQL
☐ Derived queries	☐ @Query	☐ Specifications
☐ Criteria API	☐ Pagination	☐ Sorting
☐ Transactions	☐ Persistence context	☐ Dirty checking
☐ First-level cache	☐ Second-level cache	☐ N+1 query problem
☐ Entity graphs	☐ Optimistic locking	☐ Pessimistic locking
☐ Auditing	☐ Flyway or Liquibase migrations	

31 Spring Security

KEY REVISION NOTES

- **Filter chain**: a servlet `DelegatingFilterProxy` hands requests to Spring's `FilterChainProxy`, which runs a `SecurityFilterChain` of filters (CORS, CSRF, authentication, exception translation, authorization). Configure it with a `SecurityFilterChain` bean; `WebSecurityConfigurerAdapter` has been removed.
- **Authentication flow**: an authentication filter extracts credentials → `AuthenticationManager` → an `AuthenticationProvider` → `UserDetailsService.loadUserByUsername` → `PasswordEncoder.matches` → a successful `Authentication` is stored in the `SecurityContextHolder` (thread-local) for the request.
- **Passwords**: store only salted, slow, adaptive hashes such as BCrypt (or Argon2). Use `PasswordEncoderFactories.createDelegatingPasswordEncoder()`. Never store plain text or fast hashes like MD5/SHA-1.
- **Authorization**: URL rules with `authorizeHttpRequests` and method security with `@EnableMethodSecurity` + `@PreAuthorize("hasRole('ADMIN')")` or SpEL like `#id == authentication.principal.id`. `hasRole('ADMIN')` checks the authority `ROLE_ADMIN`; `hasAuthority` checks the exact string.
- **JWT**: `header.payload.signature`, Base64URL-encoded and signed (not encrypted), so never put secrets in it. The server verifies the signature and expiry on each request, which keeps it stateless. Use short-lived access tokens plus refresh tokens with rotation; revocation requires a denylist or short expiry.

- **OAuth 2.0 vs OpenID Connect**: OAuth 2.0 is delegated authorization (an access token to call APIs), with the Authorization Code + PKCE flow for user-facing apps and Client Credentials for service-to-service. OIDC adds authentication on top (an ID token describing the user). Spring supports OAuth login (client) and resource servers that validate JWTs from providers like Keycloak, Okta or Auth0.
- **CSRF**: an attack where a malicious site makes the browser send an authenticated request using cookies. Keep CSRF protection on for session/cookie auth; it's commonly disabled for stateless APIs that use bearer tokens in headers.
- **CORS** must be configured in Spring Security too (`http.cors(...)`) so preflight requests are not rejected.
- **Session vs stateless**: sessions are simple for server-rendered apps; stateless tokens scale horizontally across instances. Set `SessionCreationPolicy.STATELESS` for JWT APIs.
- **Testing and secrets**: test with `@WithMockUser` and `spring-security-test`. Keep keys and passwords in environment variables, Vault or a cloud secrets manager, never in Git.

CODE · STATELESS RESOURCE SERVER CONFIG

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())                   // stateless token API
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**", "/actuator/health").permitAll()
            .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}

@Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Walk through how JWT authentication works end to end.
Q2 OAuth 2.0 vs OpenID Connect?
Q3 Why is CSRF often disabled for REST APIs? When must it stay enabled?
Q4 hasRole vs hasAuthority?

TOPIC CHECKLIST · 19 ITEMS

☐ Authentication and authorization	☐ Security filter chain	☐ Password hashing
☐ BCrypt	☐ UserDetailsService	☐ Role-based access control
☐ Method-level security	☐ JWT authentication	☐ Access and refresh tokens
☐ OAuth 2.0	☐ OpenID Connect	☐ OAuth login
☐ Resource servers	☐ CORS	☐ CSRF
☐ Session-based authentication	☐ Stateless authentication	☐ Security testing
☐ Secrets management		

32 Spring Transactions

KEY REVISION NOTES

- **How @Transactional works**: Spring wraps the bean in a proxy. The proxy starts a transaction before the method, commits on normal return and rolls back on failure. Put it on service-layer methods, not controllers.
- **Rollback rules**: by default only unchecked exceptions (`RuntimeException`) and `Error` cause rollback. Checked exceptions commit unless you specify `rollbackFor = Exception.class`.
- **Classic pitfalls**: self-invocation (calling a `@Transactional` method from another method in the same class bypasses the proxy); non-public methods (not proxied in many setups); catching and swallowing the exception (nothing to roll back); the object not being a Spring bean; `REQUIRES_NEW` inside the same class (also self-invocation).
- **Isolation**: `@Transactional(isolation = Isolation.REPEATABLE_READ)`; `DEFAULT` uses the database's level.
- **readOnly = true**: a hint that lets Hibernate skip dirty checking and flushing, and can route queries to read replicas. Use it on query methods.
- **Keep transactions short**: don't make HTTP calls, send emails or wait on user input inside a transaction; it holds DB connections and locks. Use `@TransactionalEventListener(phase = AFTER_COMMIT)` for side effects.
- **Distributed transactions**: two-phase commit across services is slow and fragile, so microservices use other patterns. A **Saga** is a sequence of local transactions where each failure triggers compensating actions; it can be choreographed (services react to events) or orchestrated (a central coordinator).
- **Outbox pattern**: solves the dual-write problem (updating the DB and publishing a message atomically). Write the business change and an event row into an outbox table in the same local transaction; a relay or CDC tool (Debezium) publishes the events to Kafka.

Propagation	Behaviour
REQUIRED (default)	Join the current transaction, or start a new one if none exists
REQUIRES_NEW	Suspend the current transaction and always start a new, independent one (e.g. audit log that must persist)
NESTED	Run within a savepoint of the current transaction; can roll back to the savepoint
SUPPORTS	Join if one exists, otherwise run without a transaction
MANDATORY	Must run inside an existing transaction, otherwise throw
NOT_SUPPORTED	Suspend any current transaction and run without one
NEVER	Throw if a transaction exists

CODE · THE SELF-INVOCATION TRAP

```
@Service
class ReportService {
    public void generateAll() {
        for (var id : ids) generate(id);     // calls 'this', NOT the proxy:
    }                                        // no transaction is started!

    @Transactional
    public void generate(long id) { ... }
}
// Fixes: move generate() to another bean, or make generateAll() transactional,
// or inject the bean itself lazily / use TransactionTemplate.
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Why might @Transactional not work? List the common causes.
Q2 REQUIRED vs REQUIRES_NEW with a real use case?
Q3 Does a checked exception roll back a transaction by default?
Q4 Explain the Saga and Outbox patterns.

TOPIC CHECKLIST · 11 ITEMS

☐ @Transactional	☐ Transaction boundaries	☐ Propagation
☐ Isolation levels	☐ Rollback rules	☐ Read-only transactions
☐ Proxy-based transaction management	☐ Self-invocation pitfalls	☐ Distributed transaction challenges
☐ Saga pattern	☐ Outbox pattern	

33 Testing Java Applications

KEY REVISION NOTES

- **Test pyramid**: many fast unit tests, fewer integration tests, very few end-to-end tests. Tests should be fast, independent, repeatable and readable (Arrange / Act / Assert or Given / When / Then).
- **JUnit 5**: `@Test`, `@BeforeEach`/`@AfterEach`, `@BeforeAll`/`@AfterAll` (static), `@DisplayName`, `@Nested`, `@Disabled`, `@Tag`. Assertions: `assertEquals`, `assertThrows`, `assertAll`, `assertTimeout`. AssertJ (`assertThat(x).isEqualTo(...)`) gives fluent, readable assertions.
- **Parameterized tests**: `@ParameterizedTest` with `@ValueSource`, `@CsvSource`, `@MethodSource`, `@EnumSource` run one test with many inputs.
- **Mockito**: `@Mock` creates a mock, `@InjectMocks` injects mocks into the class under test, `when(...).thenReturn(...)` / `thenThrow` stubs behaviour, `verify(mock).method(...)` checks interactions, `ArgumentCaptor` captures arguments, `@Spy` wraps a real object. Use `@ExtendWith(MockitoExtension.class)`.
- **Test doubles**: dummy (passed but unused), stub (canned answers), spy (real object that records calls), mock (pre-programmed with expectations), fake (working lightweight implementation such as an in-memory repository).
- **Spring test slices**: `@WebMvcTest` loads only the web layer and gives you `MockMvc` (mock services with `@MockitoBean`, which replaced `@MockBean` in Boot 3.4); `@DataJpaTest` loads JPA components with rollback after each test; `@SpringBootTest` loads the full context for integration tests.
- **Testcontainers**: runs real PostgreSQL, Kafka or Redis in Docker during tests, catching issues an in-memory H2 database hides. Boot 3.1+ supports `@ServiceConnection` to wire containers automatically.
- **API and contract testing**: MockMvc or `WebTestClient` / REST Assured for HTTP tests; contract testing (Spring Cloud Contract, Pact) verifies that producers and consumers agree on an API.
- **Coverage and TDD**: JaCoCo measures coverage; high coverage does not guarantee good tests. TDD cycle: Red (failing test) → Green (simplest code that passes) → Refactor.

CODE · UNIT TEST WITH MOCKITO

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository repo;
    @Mock PaymentClient payments;
    @InjectMocks OrderService service;

    @Test
    void failsWhenPaymentDeclined() {
        when(payments.charge(any())).thenReturn(PaymentResult.DECLINED);

        assertThrows(PaymentDeclinedException.class,
                     () -> service.place(new CreateOrderRequest(1L, 2)));

        verify(repo, never()).save(any());
    }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Unit test vs integration test? What do you mock?
Q2 @Mock vs @MockitoBean (@MockBean)? Mock vs spy?
Q3 Why prefer Testcontainers over H2 for repository tests?
Q4 How do you test a controller in isolation?

TOPIC CHECKLIST · 20 ITEMS

☐ Unit testing	☐ Integration testing	☐ JUnit 5
☐ Assertions	☐ Parameterized tests	☐ Lifecycle methods
☐ Mockito	☐ Mocking and stubbing	☐ Argument captors
☐ Test doubles	☐ Spring Boot testing	☐ @SpringBootTest
☐ @WebMvcTest	☐ @DataJpaTest	☐ MockMvc
☐ Testcontainers	☐ REST API testing	☐ Contract testing
☐ Test coverage	☐ TDD	

34 Microservices

KEY REVISION NOTES

- **Monolith vs microservices**: a monolith is simpler to build, test, deploy and debug. Microservices give independent deployment and scaling, team autonomy and fault isolation, but add network failures, data consistency challenges, operational overhead and harder debugging. A well-structured modular monolith is often the right start.
- **Decomposition**: split by business capability using Domain-Driven Design bounded contexts (Orders, Payments, Inventory), not by technical layer. Each service owns its data (database per service); no sharing tables.
- **API Gateway** (Spring Cloud Gateway, Kong, cloud gateways): single entry point for routing, authentication, rate limiting, CORS and request aggregation.
- **Service discovery and config**: services find each other through a registry (Eureka, Consul) or Kubernetes DNS. Centralized configuration via Spring Cloud Config, Kubernetes ConfigMaps or a cloud parameter store.
- **Communication**: synchronous REST (OpenFeign declarative clients, `RestClient`, `WebClient`) or gRPC is simple but couples availability. Asynchronous messaging (Kafka, RabbitMQ) decouples services and absorbs load spikes but brings eventual consistency.
- **Resilience patterns** (Resilience4j): always set timeouts; retry transient failures with exponential backoff and jitter (only for idempotent operations); circuit breakers stop calling a failing service; bulkheads isolate resources so one slow dependency can't exhaust all threads; provide fallbacks.
- **Circuit breaker states**: CLOSED (calls pass, failures counted) → OPEN (calls fail fast after the threshold is exceeded) → HALF_OPEN (a few trial calls; success closes it, failure reopens it).
- **Observability across services**: distributed tracing propagates a trace ID and span IDs through every hop (Micrometer Tracing, OpenTelemetry); put correlation IDs in logs via MDC; use centralized logging.
- **Data consistency**: use sagas and the outbox pattern instead of distributed transactions; design idempotent consumers (deduplicate by message or request ID) because messages may be delivered more than once; accept eventual consistency where the business allows it.

INTERVIEW CHECK · can you answer these out loud?

Q1 When would you NOT choose microservices?
Q2 Explain the circuit breaker states.
Q3 How do you keep data consistent across services?
Q4 Sync vs async communication: trade-offs?

TOPIC CHECKLIST · 29 ITEMS

☐ Monolith vs microservices	☐ Service decomposition	☐ Domain-Driven Design fundamentals
☐ Bounded contexts	☐ API Gateway	☐ Service discovery
☐ Centralized configuration	☐ Inter-service communication	☐ REST communication
☐ OpenFeign	☐ WebClient	☐ Sync vs async communication
☐ Messaging	☐ Kafka	☐ RabbitMQ
☐ Resilience patterns	☐ Timeouts	☐ Retries
☐ Circuit breakers	☐ Bulkheads	☐ Resilience4j
☐ Distributed tracing	☐ Correlation IDs	☐ Centralized logging
☐ Distributed transactions	☐ Saga pattern	☐ Event-driven architecture
☐ Idempotent consumers	☐ Eventual consistency	

35 Caching

KEY REVISION NOTES

- **Why cache**: serve frequent reads faster and reduce load on databases and downstream services. Cache data that is read often, changes rarely and is expensive to compute.
- **Strategies**: cache-aside (the app checks the cache, on a miss loads from the DB and populates the cache; the most common approach); read-through (the cache itself loads on a miss); write-through (write to cache and DB synchronously; consistent but slower writes); write-behind (write to cache, persist to DB asynchronously; fast but risks data loss).
- **Expiry and eviction**: TTL limits staleness. Eviction policies such as LRU (least recently used) and LFU (least frequently used) decide what to drop when full.
- **Local vs distributed**: a local in-memory cache (Caffeine) is fastest but each instance has its own copy, which can diverge. A distributed cache (Redis) is shared by all instances and survives restarts but adds a network hop. Some systems use both (two-level caching).
- **Spring Cache abstraction**: `@EnableCaching`, `@Cacheable(value = "products", key = "#id")` reads and populates, `@CachePut` always runs and updates the cache, `@CacheEvict` removes entries (`allEntries = true` clears all). It is proxy-based, so self-invocation is not cached.
- **Failure modes**: cache stampede / thundering herd (a hot key expires and many requests hit the DB at once; fix with locking or request coalescing, early refresh, or jittered TTLs); cache penetration (repeated requests for non-existent keys; cache empty results briefly or use a Bloom filter); cache avalanche (many keys expire together; randomise TTLs).
- **Consistency**: common approach is update the DB first, then delete (invalidate) the cache entry, accepting a short window of staleness. Decide explicitly how stale data may be.
- **Redis beyond caching**: data types (strings, hashes, lists, sets, sorted sets for leaderboards), distributed locks, rate limiting counters, session storage, pub/sub.

CODE · SPRING CACHE WITH REDIS OR CAFFEINE

```
@Service
class ProductService {
    @Cacheable(value = "products", key = "#id")
    public ProductDto get(long id) { return repo.findById(id).map(ProductDto::from).orElseThrow(); }

    @CachePut(value = "products", key = "#result.id")
    public ProductDto update(UpdateProductRequest req) { ... }

    @CacheEvict(value = "products", key = "#id")
    public void delete(long id) { repo.deleteById(id); }
}
# application.yml: spring.cache.type=redis  and  spring.cache.redis.time-to-live=10m
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Cache-aside vs write-through vs write-behind?
Q2 How do you avoid serving stale data?
Q3 What is a cache stampede and how do you prevent it?

TOPIC CHECKLIST · 16 ITEMS

- ☐ Cache fundamentals
- ☐ Cache-aside
- ☐ Read-through and write-through
- ☐ Write-behind
- ☐ TTL
- ☐ Cache invalidation
- ☐ Local caching
- ☐ Caffeine
- ☐ Redis
- ☐ Spring Cache abstraction
- ☐ @Cacheable
- ☐ @CachePut
- ☐ @CacheEvict
- ☐ Distributed caching
- ☐ Cache stampede
- ☐ Cache consistency

36 Messaging and Event Streaming

KEY REVISION NOTES

- **Queue vs topic**: a queue is point-to-point (each message is handled by one consumer). A topic is publish/subscribe (every subscriber gets every message).
- **Kafka basics**: producers write records to topics; topics are split into partitions; the record key determines the partition, so all events for one key (e.g. an order ID) stay ordered. Consumers in the same consumer group share partitions (each partition goes to one member), so consumers beyond the partition count sit idle. Consumers commit offsets to track progress.
- **RabbitMQ basics**: producers publish to an exchange (direct, topic, fanout, headers), which routes messages to queues by binding keys. Consumers acknowledge messages; unacknowledged ones are redelivered.
- **Delivery semantics**: at-most-once (commit before processing, may lose messages); at-least-once (process then commit, may produce duplicates; the usual choice); exactly-once (Kafka's idempotent producer and transactions give it within Kafka, but end-to-end it still needs idempotent processing).
- **Idempotent consumers**: store processed message IDs or use upserts so a redelivered message has no extra effect.
- **Failure handling**: retries with backoff, then route poison messages to a dead-letter queue/topic for inspection instead of blocking the stream.
- **Event schemas**: Avro, Protobuf or JSON Schema with a Schema Registry enforcing backward/forward compatibility so producers and consumers can evolve independently.
- **Outbox and CDC**: the outbox pattern ensures events are published only when the DB transaction commits. Change Data Capture (Debezium) reads the database's transaction log (WAL/binlog) and streams changes to Kafka.

Aspect	Kafka	RabbitMQ
Model	Distributed append-only log	Message broker with exchanges and queues
Retention	Kept for a configured time; consumers can replay	Removed once acknowledged
Ordering	Guaranteed within a partition	Per queue (with a single consumer)
Scaling	Partitions + consumer groups; very high throughput	Competing consumers; flexible routing
Best for	Event streaming, event sourcing, analytics pipelines, high volume	Task queues, complex routing, request/reply

CODE · SPRING KAFKA PRODUCER AND CONSUMER

```
kafkaTemplate.send("order-events", order.id().toString(), new OrderPlaced(order.id()));

@KafkaListener(topics = "order-events", groupId = "notification-service")
public void on(OrderPlaced event) {
    if (processed.exists(event.eventId())) return;   // idempotent consumer
    emailService.sendConfirmation(event.orderId());
    processed.save(event.eventId());
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Kafka vs RabbitMQ: when would you pick each?
Q2 How does Kafka guarantee ordering? What happens with more consumers than partitions?
Q3 How do you handle duplicate and poison messages?

TOPIC CHECKLIST · 17 ITEMS

☐ Message queues	☐ Topics and partitions	☐ Producers and consumers
☐ Delivery semantics	☐ At-most-once	☐ At-least-once
☐ Exactly-once concepts	☐ Consumer groups	☐ Kafka
☐ RabbitMQ	☐ Dead-letter queues	☐ Retry strategies
☐ Event schemas	☐ Schema Registry	☐ Event ordering
☐ Outbox pattern	☐ Change Data Capture	

37 Observability

KEY REVISION NOTES

- **Three pillars**: logs (discrete events with context), metrics (numeric measurements aggregated over time), traces (the path of one request across services). Together they answer what happened, how much, and where.
- **Logging**: structured JSON logs that include a trace ID; correct levels (ERROR for failures needing action, WARN, INFO for business events, DEBUG for diagnostics); never log passwords, tokens or personal data. MDC (Mapped Diagnostic Context) adds per-request fields like user or correlation ID.
- **Micrometer**: a vendor-neutral metrics facade (like SLF4J for metrics) built into Spring Boot. Meter types: counter (only goes up), gauge (current value), timer (latency and count), distribution summary. Prometheus scrapes `/actuator/prometheus`; Grafana visualises dashboards.
- **Tracing**: OpenTelemetry is the vendor-neutral standard for traces, metrics and logs. Micrometer Tracing bridges to it. Backends include Jaeger, Zipkin and Grafana Tempo. Log aggregation with Loki or the ELK stack.
- **Health probes** (Kubernetes): liveness (is the process stuck? if it fails, restart the container), readiness (can it serve traffic? if it fails, remove it from the load balancer), startup (give slow-starting apps time). Don't make liveness depend on external systems like the database.
- **What to measure**: RED for services (Rate, Errors, Duration) and USE for resources (Utilisation, Saturation, Errors). Track latency percentiles (p50, p95, p99), not just averages.
- **SLI, SLO, SLA**: an SLI is a measured indicator (e.g. % of requests under 300 ms); an SLO is the internal target (99.9% over 30 days); an SLA is a contract with customers and penalties. The error budget (100% − SLO) tells you how much risk you can take.
- **Alerting**: alert on user-facing symptoms (error rate, latency SLO burn), not every CPU spike; every alert should be actionable to avoid alert fatigue.

CODE · CUSTOM METRICS WITH MICROMETER

```
@Component
class OrderMetrics {
    private final Counter placed;
    private final Timer checkout;
    OrderMetrics(MeterRegistry registry) {
        placed = Counter.builder("orders.placed").tag("channel", "web").register(registry);
        checkout = Timer.builder("orders.checkout.duration").publishPercentiles(0.95, 0.99).register(registry);
    }
    void record(Runnable r) { checkout.record(r); placed.increment(); }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Liveness vs readiness probes?
Q2 Explain SLI, SLO and SLA with an example.
Q3 How would you debug a slow request that crosses several services?

TOPIC CHECKLIST · 15 ITEMS

☐ Application logging	☐ Structured logs	☐ Metrics
☐ Distributed tracing	☐ Spring Boot Actuator	☐ Micrometer
☐ Prometheus	☐ Grafana	☐ OpenTelemetry
☐ Jaeger	☐ Log aggregation	☐ Loki
☐ Health and readiness probes	☐ Alerting	☐ SLI, SLO and SLA

38 Docker, CI/CD and Cloud

KEY REVISION NOTES

- **Linux essentials**: `ls`, `cd`, `cat`, `less`, `grep`, `tail -f`, `find`, `ps aux`, `top`/`htop`, `kill`, `chmod`/`chown`, `df -h`, `free -m`, `curl`, `ss -tulpn`, `env`/`export`, `systemctl`, `journalctl`. Environment variables configure apps without code changes.
- **Image vs container**: an image is an immutable, layered template; a container is a running instance of an image with its own filesystem layer, network and process space.
- **Dockerfile best practices**: use a small JRE base image (e.g. Eclipse Temurin), multi-stage builds (build with JDK + Maven, run with JRE only), order layers for caching (copy `pom.xml` and download dependencies before copying source), run as a non-root user, add a `.dockerignore`, and use container-aware JVM settings like `-XX:MaxRAMPercentage`.
- **CMD vs ENTRYPOINT**: `ENTRYPOINT` defines the executable; `CMD` provides default arguments that can be overridden at `docker run`.
- **Docker Compose**: define multi-container setups (app + PostgreSQL + Redis + Kafka) for local development. Containers on the same network reach each other by service name; volumes persist data beyond the container's life.
- **CI/CD**: continuous integration builds and tests every push and pull request. Continuous delivery keeps every build deployable with a manual release step; continuous deployment releases automatically to production. Artifacts (JARs, images) are pushed to a registry.
- **GitHub Actions**: workflows in `.github/workflows/*.yml` triggered by events; jobs run on runners; steps run actions or shell commands; use secrets for credentials and caching for Maven dependencies.
- **AWS fundamentals**: EC2 (virtual machines), S3 (object storage), RDS (managed relational DB), IAM (users, roles, policies with least privilege; services use roles, not keys), CloudWatch (logs, metrics, alarms), Secrets Manager, ALB/NLB (load balancers), ECS/EKS (containers), Lambda (serverless).
- **Reverse proxy and load balancer**: Nginx or a cloud load balancer terminates TLS, routes by host or path, balances traffic and can cache or rate limit.
- **Kubernetes basics**: Pod (one or more containers), Deployment (desired replicas, rolling updates), Service (stable network endpoint), Ingress (HTTP routing), ConfigMap and Secret (configuration), HorizontalPodAutoscaler (scale on metrics).

- **Deployment strategies**: rolling (replace instances gradually), blue-green (switch traffic between two full environments; instant rollback), canary (send a small % of traffic to the new version first). Always have a rollback plan and backward-compatible DB migrations.

CODE · MULTI-STAGE DOCKERFILE + CI WORKFLOW

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:21-jre
RUN useradd -r app
USER app
COPY --from=build /app/target/*.jar /app.jar
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]

# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: ./mvnw -B verify
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Docker image vs container? CMD vs ENTRYPOINT?
Q2 Continuous delivery vs continuous deployment?
Q3 Blue-green vs canary deployments?
Q4 How would you deploy a Spring Boot app with a database to AWS?

TOPIC CHECKLIST · 30 ITEMS

☐ Linux fundamentals	☐ Shell commands	☐ Process management
☐ Environment variables	☐ Docker images	☐ Docker containers
☐ Dockerfiles	☐ Docker Compose	☐ Container networking
☐ Persistent volumes	☐ Multi-stage builds	☐ GitHub Actions
☐ Build pipelines	☐ Automated testing	☐ Continuous integration
☐ Continuous delivery/deployment	☐ Artifact repositories	☐ AWS fundamentals
☐ EC2	☐ S3	☐ IAM
☐ RDS	☐ CloudWatch	☐ Secrets Manager
☐ Load balancers	☐ Reverse proxies	☐ Kubernetes fundamentals
☐ Deployment strategies	☐ Rollbacks	☐ Production configuration

39 System Design

KEY REVISION NOTES

- **Interview framework**: (1) clarify functional requirements, (2) non-functional requirements (scale, latency, availability, consistency), (3) back-of-envelope estimates (users, QPS, storage, bandwidth), (4) API design, (5) data model, (6) high-level architecture, (7) deep dive into bottlenecks, (8) discuss trade-offs and failure handling.
- **Scaling**: vertical (bigger machine; simple but limited) vs horizontal (more machines; requires stateless services with sessions and files stored externally).
- **Performance terms**: latency (time per request) vs throughput (requests per second). Measure percentiles (p99) because averages hide slow outliers. Reliability means working correctly; availability means being reachable; fault tolerance means continuing despite failures.
- **Load balancing**: L4 (TCP) vs L7 (HTTP, can route by path or header). Algorithms: round robin, least connections, IP hash, consistent hashing (minimises remapping when nodes change).
- **Database scaling path**: optimise queries and indexes → add caching → read replicas (replication for read scaling and failover, with replication lag) → partitioning and sharding (split data by a shard key; choose keys that avoid hotspots; cross-shard queries are hard) → CQRS or specialised stores (search engine, time-series DB).
- **CAP theorem**: during a network partition a distributed system must choose consistency (refuse or delay requests) or availability (respond with possibly stale data). PACELC adds: else, choose between latency and consistency.
- **Consistency models**: strong (every read sees the latest write), eventual (replicas converge over time), read-your-writes, monotonic reads.
- **Rate limiting algorithms**: token bucket (allows bursts, most common), leaky bucket (smooth output), fixed window counter (simple, burst at edges), sliding window log/counter (more accurate). Usually enforced at the gateway with Redis counters.
- **Robustness**: timeouts everywhere, retries with exponential backoff + jitter for idempotent calls, idempotency keys, circuit breakers, queues to absorb spikes, graceful degradation.
- **Practice problems**: URL shortener, rate limiter, notification service, chat application, news feed, file storage, e-commerce checkout, ride matching. Low-level design (LLD): parking lot, elevator, library system, splitwise, BookMyShow. Draw UML class and sequence diagrams.

Availability	Downtime per year	Downtime per month
99% (two nines)	~3.65 days	~7.3 hours
99.9% (three nines)	~8.76 hours	~43.8 minutes
99.99% (four nines)	~52.6 minutes	~4.4 minutes
99.999% (five nines)	~5.3 minutes	~26 seconds

INTERVIEW CHECK · can you answer these out loud?

Q1 Design a URL shortener (estimates, ID generation, storage, caching, redirects).
Q2 Explain the CAP theorem with a real example.
Q3 Replication vs sharding? How do you choose a shard key?
Q4 How would you design a rate limiter?

TOPIC CHECKLIST · 27 ITEMS

☐ Requirements gathering	☐ Functional vs non-functional	☐ Scalability
☐ Availability	☐ Reliability	☐ Latency and throughput
☐ Horizontal vs vertical scaling	☐ Load balancing	☐ Caching
☐ Database scaling	☐ Replication	☐ Sharding
☐ Message queues	☐ Rate limiting	☐ API Gateway
☐ Monolith vs microservices	☐ Consistency models	☐ CAP theorem
☐ High availability	☐ Fault tolerance	☐ Idempotency
☐ Retry and timeout design	☐ Database transactions	☐ Observability
☐ Security design	☐ Design patterns	☐ UML and sequence diagrams

PART C

Advanced Java Topics

Design patterns, data structures and algorithms, JVM internals and specialised APIs (topics 40-43).

40 Design Patterns

KEY REVISION NOTES

- **Creational** patterns control how objects are created (Singleton, Factory Method, Abstract Factory, Builder, Prototype).
- **Structural** patterns compose classes and objects (Adapter, Decorator, Facade, Proxy, Composite, Bridge separates abstraction from implementation, Flyweight shares state).
- **Behavioral** patterns manage responsibilities and communication (Strategy, Observer, Command, Template Method, State changes behaviour with internal state, Chain of Responsibility, Iterator, Mediator centralises communication, Visitor adds operations to a class hierarchy without changing it).
- **Thread-safe Singleton**: the simplest is an `enum` (also safe against serialization and reflection). Alternatives are the holder-class idiom (lazy, no locking) or double-checked locking, which requires a `volatile` field.
- **Decorator vs Proxy**: both wrap an object with the same interface. A decorator adds behaviour (and can stack); a proxy controls access (lazy loading, security, transactions, remote calls).
- **Strategy vs State**: Strategy is chosen by the client and stays fixed for an operation; State changes automatically as the object's internal state changes.
- **Enterprise patterns**: Repository (collection-like access to data), Unit of Work (track changes and commit together, i.e. the JPA persistence context), Dependency Injection, MVC, Front Controller (`DispatcherServlet`), DTO, Service Layer, Circuit Breaker.
- **Use patterns to solve real problems**, not to show off. Explain the problem a pattern solved in one of your projects.

Pattern	Intent	Where you see it in Java/Spring
Singleton	One instance	Spring singleton beans, Runtime.getRuntime()
Factory Method / Abstract Factory	Decouple object creation	Calendar.getInstance(), BeanFactory, List.of()
Builder	Step-by-step construction, many optional fields	StringBuilder, HttpClient.newBuilder(), Lombok @Builder
Prototype	Copy an existing object	clone(), Spring prototype scope
Adapter	Convert one interface into another	InputStreamReader, Arrays.asList(), HandlerAdapter
Decorator	Add behaviour by wrapping	BufferedInputStream, Collections.unmodifiableList()
Proxy	Control access to an object	Spring AOP, @Transactional, Hibernate lazy proxies
Facade	Simple interface over a complex subsystem	JdbcTemplate, SLF4J
Composite	Treat part-whole trees uniformly	UI component trees, nested validators
Flyweight	Share instances to save memory	Integer cache, String pool
Strategy	Swap algorithms at runtime	Comparator, PasswordEncoder implementations
Observer	Notify subscribers of changes	Spring ApplicationEvent / @EventListener
Template Method	Fixed algorithm skeleton, custom steps	JdbcTemplate, AbstractList, RestTemplate
Chain of Responsibility	Pass a request along handlers	Servlet filters, Spring Security filter chain
Command	Encapsulate a request as an object	Runnable, Callable, task queues
Iterator	Traverse without exposing internals	java.util.Iterator

CODE · SINGLETON OPTIONS

```
public enum IdGenerator {                 // simplest thread-safe singleton
    INSTANCE;
    private final AtomicLong next = new AtomicLong();
    public long nextId() { return next.incrementAndGet(); }
}

public final class Config {               // lazy holder idiom
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    public static Config getInstance() { return Holder.INSTANCE; }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Which design patterns does Spring use? Give examples.
Q2 How do you make a Singleton thread-safe? Why is enum the best option?
Q3 Decorator vs Proxy? Strategy vs State?

TOPIC CHECKLIST · 29 ITEMS

☐ Singleton	☐ Factory Method	☐ Abstract Factory
☐ Builder	☐ Prototype	☐ Adapter
☐ Decorator	☐ Facade	☐ Proxy
☐ Composite	☐ Bridge	☐ Flyweight
☐ Strategy	☐ Observer	☐ Command
☐ Template Method	☐ State	☐ Chain of Responsibility
☐ Iterator	☐ Mediator	☐ Visitor
☐ Repository	☐ Unit of Work	☐ Dependency Injection
☐ MVC	☐ Front Controller	☐ Data Transfer Object
☐ Service Layer	☐ Circuit Breaker	

41 Data Structures and Algorithms (DSA)

KEY REVISION NOTES

- **Big-O order**: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n) < O(n!). State both time and space complexity for every solution. Amortized analysis averages the cost over a sequence of operations (e.g. ArrayList add is amortized O(1) despite occasional resizing).
- **Rough limits**: for about 10^8 simple operations per second, n ≤ 10^5 usually needs O(n log n) or better, n ≤ 10^4 allows O(n^2), n ≤ 20 allows O(2^n).
- **Java toolkit**: `ArrayDeque` as stack (`push`/`pop`) and queue (`offer`/`poll`); `PriorityQueue` is a min-heap (use `Comparator.reverseOrder()` or `(a, b) -> b - a` carefully, prefer `Integer.compare` to avoid overflow); `map.merge(key, 1, Integer::sum)` and `getOrDefault` for counting; `TreeMap.floorKey`/`ceilingKey` for ordered lookups; `Arrays.fill`, `Collections.reverse`, `StringBuilder.reverse()`.
- **Sorting**: merge sort O(n log n), stable, O(n) space; quicksort average O(n log n), worst O(n^2), in-place; heap sort O(n log n), in-place, not stable; counting/radix sort for small integer ranges. Java sorts objects with stable TimSort and primitives with dual-pivot quicksort.
- **Trees**: know preorder, inorder (sorted order for a BST), postorder and level-order traversals, both recursive and iterative. Height, diameter, lowest common ancestor, validate BST, serialise/deserialise.
- **Graphs**: represent with adjacency lists (`List<List<Integer>>`). BFS uses a queue and finds shortest paths in unweighted graphs; DFS uses recursion or a stack for connectivity, cycle detection and topological order. Track visited nodes.
- **Dynamic programming**: define the state, the recurrence, base cases and the order of computation. Start with recursion + memoization (top-down), then convert to tabulation (bottom-up) and optimise space. Classics: climbing stairs, house robber, coin change, LIS, LCS, knapsack, edit distance, unique paths.

- **Advanced structures**: Union-Find with path compression and union by rank (nearly O(1)), segment trees and Fenwick trees for range queries with updates, tries for prefix problems, bit manipulation tricks (`x & (x - 1)` clears the lowest set bit, `x & -x` isolates it).
- **Practice plan**: around 150-200 problems across all patterns (curated lists such as Blind 75 or NeetCode 150), mostly Easy and Medium. Explain your approach before coding, test edge cases (empty input, one element, duplicates, negatives, overflow) and analyse complexity out loud.

If the problem mentions...	Think of...
Sorted array, find a target or boundary	Binary search (also binary search on the answer)
Pairs/triplets in a sorted array, palindromes	Two pointers
Longest/shortest contiguous subarray or substring	Sliding window
Range sums, subarray sum equals K	Prefix sums + HashMap
Counting, duplicates, fast lookup	HashMap / HashSet
Top K, Kth largest, merge K sorted lists	Heap (PriorityQueue)
Next greater/smaller element, histogram	Monotonic stack
Shortest path in an unweighted graph or grid	BFS
Shortest path with weights	Dijkstra (non-negative weights)
Dependencies / ordering of tasks	Topological sort (Kahn's algorithm)
Connected components, cycle detection in undirected graphs	Union-Find or DFS
All combinations/permutations/subsets	Backtracking
Optimal value with overlapping subproblems	Dynamic programming
Prefix/word search	Trie
Overlapping intervals, meeting rooms	Sort by start + merge / heap

CODE · TEMPLATES WORTH MEMORISING

```
// Sliding window: longest substring without repeating characters
int best = 0; Map<Character, Integer> last = new HashMap<>();
for (int l = 0, r = 0; r < s.length(); r++) {
    char c = s.charAt(r);
    if (last.containsKey(c)) l = Math.max(l, last.get(c) + 1);
    last.put(c, r);
    best = Math.max(best, r - l + 1);
}

// BFS on a grid
Deque<int[]> q = new ArrayDeque<>(); q.offer(new int[]{sr, sc}); seen[sr][sc] = true;
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : DIRS) { int r = cur[0] + d[0], c = cur[1] + d[1];
        if (r >= 0 && c >= 0 && r < R && c < C && !seen[r][c] && grid[r][c] == 1) {
            seen[r][c] = true; q.offer(new int[]{r, c}); } }
}

// Binary search (first index where condition is true)
int lo = 0, hi = n;                        // search space [lo, hi)
while (lo < hi) { int mid = lo + (hi - lo) / 2; if (ok(mid)) hi = mid; else lo = mid + 1; }
```

INTERVIEW CHECK · can you answer these out loud?

Q1 Why is `lo + (hi - lo) / 2` used instead of `(lo + hi) / 2`?
Q2 Why does Java use a stable sort for objects but not for primitives?
Q3 Explain how you'd recognise that a problem needs dynamic programming.

TOPIC CHECKLIST · 36 ITEMS

☐ Big-O notation	☐ Time complexity	☐ Space complexity
☐ Amortized analysis	☐ Arrays	☐ Strings
☐ Linked lists	☐ Stacks	☐ Queues
☐ Hash tables	☐ Trees	☐ Binary search trees
☐ Heaps	☐ Tries	☐ Graphs
☐ Union-Find	☐ Segment trees	☐ Fenwick trees
☐ Sorting	☐ Binary search	☐ Two pointers
☐ Sliding window	☐ Prefix sums	☐ Hashing
☐ Recursion	☐ Backtracking	☐ Tree traversal
☐ Graph traversal	☐ BFS and DFS	☐ Topological sorting
☐ Shortest paths	☐ Greedy algorithms	☐ Dynamic programming
☐ Monotonic stacks	☐ Intervals	☐ Bit manipulation

42 Advanced Java Internals

KEY REVISION NOTES

- **Bytecode inspection**: `javap -c -p MyClass` disassembles bytecode. Invocation instructions: `invokestatic`, `invokespecial` (constructors, private, super), `invokevirtual` (class methods), `invokeinterface`, and `invokedynamic` (used for lambdas, string concatenation since Java 9, and records' generated methods).
- **Java Memory Model in depth**: without happens-before relationships, the compiler and CPU may reorder instructions and threads may see stale values. Safe publication: final fields set in the constructor, volatile, locks, or concurrent collections.
- **Class initialization**: happens lazily on first active use (creating an instance, calling a static method, accessing a non-constant static field), and the JVM guarantees it runs exactly once in a thread-safe way, which is why the holder idiom works.
- **Method handles and VarHandle**: `MethodHandles.lookup()` gives faster, type-safe alternatives to reflection. `VarHandle` offers atomic and ordered field access (compare-and-set, acquire/release) and replaces most uses of `sun.misc.Unsafe`.
- **Java agents and Instrumentation**: an agent (`-javaagent:agent.jar`, with `premain`, or attached at runtime with `agentmain`) can transform bytecode as classes load. APM tools (New Relic, OpenTelemetry Java agent), profilers and Mockito's inline mock maker use this.
- **Annotation processors** generate code at compile time (Lombok, MapStruct); no runtime cost.
- **JNI vs FFM API**: JNI is the legacy, error-prone way to call native C code. The Foreign Function & Memory API (final in Java 22) provides a safer, pure-Java way using `Linker`, `MemorySegment` and `Arena`.
- **Custom class loaders**: used for plugin systems, application servers and hot reloading. A class's identity is its name plus its class loader, so the same class loaded twice gives two incompatible types; class loader leaks cause Metaspace OOMs.
- **GC tuning**: first set the goal (latency vs throughput vs footprint), right-size the heap, choose the collector, then read GC logs (`-Xlog:gc*`). Measure before and after; most applications need only heap sizing and a suitable collector.
- **JMH**: the Java Microbenchmark Harness handles JIT warm-up, dead-code elimination and constant folding, which make naive `System.nanoTime()` benchmarks misleading. Use `@Benchmark`, `Blackhole`, warm-up and measurement iterations.

CODE · INSPECTING BYTECODE AND A JMH BENCHMARK

```
javap -c -p Greeter.class      # look for invokedynamic for lambdas/concat

@State(Scope.Benchmark)
public class ConcatBench {
    String a = "hello", b = "world";
    @Benchmark public String plus() { return a + b; }
    @Benchmark public String builder() { return new StringBuilder(a).append(b).toString(); }
}
```

INTERVIEW CHECK · can you answer these out loud?

Q1 What is invokedynamic and where is it used?
Q2 Why are naive microbenchmarks misleading and how does JMH help?
Q3 How can two classes with the same fully qualified name be incompatible?

TOPIC CHECKLIST · 18 ITEMS

☐ Java Memory Model	☐ Happens-before guarantees	☐ Class loading and initialization
☐ Bytecode inspection	☐ javap	☐ Reflection internals
☐ Method handles	☐ VarHandle	☐ Garbage collection tuning
☐ JIT compilation	☐ Java agents	☐ Instrumentation API
☐ Annotation processors	☐ JNI	☐ Foreign Function and Memory API
☐ Custom class loaders	☐ Module system internals	☐ Benchmarking with JMH

43 Additional Specialized Topics

KEY REVISION NOTES

- **Networking**: TCP is connection-oriented, reliable and ordered; UDP is connectionless, faster, with no delivery guarantee (video, DNS, games). Java offers `Socket`/`ServerSocket` for TCP and `DatagramSocket` for UDP.
- **HTTP Client API (Java 11)**: `java.net.http.HttpClient` supports HTTP/2, sync (`send`) and async (`sendAsync` returning `CompletableFuture`) requests, and works well with virtual threads.
- **RMI**: legacy remote method invocation between JVMs; replaced by REST, gRPC and messaging.
- **Reactive programming**: Reactive Streams defines `Publisher`, `Subscriber`, `Subscription` and `Processor` with backpressure (the consumer controls how much data it receives). Project Reactor provides `Mono` (0 or 1 item) and `Flux` (0..N items). Spring WebFlux runs on non-blocking Netty. It suits streaming and very high concurrency, but adds complexity; virtual threads now cover many of the same scalability needs with simple blocking code.
- **GraalVM native image**: ahead-of-time compilation into a native executable with fast startup and low memory (good for serverless and CLIs). Trade-offs: longer builds, closed-world assumption, reflection and proxies need configuration (Spring Boot AOT generates most of it), and peak throughput may be lower than the JIT.
- **Desktop**: Swing (older, part of the JDK) and JavaFX (modern, separate OpenJFX library) for desktop UIs; uncommon in backend roles.
- **Internationalization (i18n) and localization (l10n)**: `Locale`, `ResourceBundle`, `MessageFormat`, `NumberFormat`, and Spring's `MessageSource` for translated messages.
- **Cryptography and secure coding**: `MessageDigest` (SHA-256 hashing), `Cipher` (AES-GCM for encryption), `KeyStore` (certificates and keys), `Signature` (digital signatures), `SecureRandom`. Never invent your own crypto. Follow the OWASP Top 10: validate input, use parameterised queries, encode output, apply least privilege, and keep dependencies patched.
- **Dependency vulnerability scanning**: OWASP Dependency-Check, Snyk, GitHub Dependabot and Trivy (for images), integrated into CI.

Style	Transport / format	Strengths	Typical use
REST	HTTP/1.1 or 2, JSON	Simple, cacheable, universal tooling	Public and CRUD APIs
gRPC	HTTP/2, Protobuf	Fast, strongly typed contracts, streaming	Internal service-to-service calls
GraphQL	HTTP, JSON, single endpoint	Clients choose exactly the fields they need	Flexible frontends, aggregating many sources
WebSocket	Persistent full-duplex TCP connection	Real-time server push	Chat, live dashboards, notifications

INTERVIEW CHECK · can you answer these out loud?

Q1 REST vs gRPC vs GraphQL: when would you choose each?
Q2 Mono vs Flux? What is backpressure?
Q3 When is a GraalVM native image worth it?

TOPIC CHECKLIST · 22 ITEMS

- ☐ JavaFX desktop applications
- ☐ Swing
- ☐ Java networking
- ☐ TCP and UDP sockets
- ☐ HTTP Client API
- ☐ RMI fundamentals
- ☐ WebSocket
- ☐ gRPC
- ☐ GraphQL Java
- ☐ Reactive programming
- ☐ Project Reactor
- ☐ Spring WebFlux
- ☐ Reactive Streams
- ☐ Native image compilation
- ☐ GraalVM
- ☐ Internationalization
- ☐ Localization
- ☐ Cryptography APIs
- ☐ Digital signatures
- ☐ Certificate management
- ☐ Secure coding
- ☐ Dependency vulnerability scanning

PART D

Projects and Priorities

Portfolio projects and what to prioritise for a Java backend job (topics 44-45).

44 Java Projects for Practical Learning

KEY REVISION NOTES

- **Quality over quantity**: two or three complete, polished projects beat ten tutorial clones. Recruiters and interviewers look for evidence that you can design, build, test, deploy and explain software.
- **Project quality checklist**: clean layered architecture; meaningful commit history; unit and integration tests (Testcontainers); global error handling and validation; OpenAPI docs; Docker Compose so it runs with one command; CI pipeline; deployed live demo; observability (Actuator, metrics, logs).
- **README essentials**: what the project does and why, tech stack, architecture diagram, how to run locally, API overview, key design decisions and trade-offs, what you would improve next.
- **Talk about it**: prepare a two-minute walkthrough of each project, the hardest bug you fixed, a design decision you'd change, and how it would scale to 100x traffic.

Level	Project	Skills it proves
Beginner	Calculator	Syntax, operators, control flow, input handling
Beginner	Student grade system	Arrays/collections, methods, validation
Beginner	Banking console application	OOP, encapsulation, custom exceptions
Beginner	Library management system	Class design, collections, streams, file persistence
Beginner	Employee management system	CRUD logic, sorting with comparators, JDBC
Intermediate	Spring Boot CRUD API	REST, layered architecture, JPA, validation
Intermediate	Employee management REST API	DTOs, pagination, global errors, OpenAPI docs
Intermediate	Authentication system	Spring Security, BCrypt, JWT, roles, refresh tokens
Intermediate	Inventory management	Transactions, optimistic locking, Flyway, Testcontainers
Intermediate	E-commerce backend	Relationships, cart/checkout flow, caching, payments mock
Advanced	Order management microservices	Service boundaries, gateway, Feign, Resilience4j
Advanced	Payment integration service	Idempotency keys, webhooks, retries, security
Advanced	Kafka event-driven system	Producers/consumers, outbox, DLQ, idempotent consumers
Advanced	Redis caching application	Cache-aside, eviction, rate limiting, stampede protection
Advanced	Production-ready SaaS backend	Multi-tenancy, CI/CD, Docker, metrics, tracing, alerts

INTERVIEW CHECK · can you answer these out loud?

Q1 Walk me through the architecture of your best project.
Q2 What was the hardest technical problem you solved in it?
Q3 How would your project handle 100x more traffic?

TOPIC CHECKLIST · 15 ITEMS

☐ Calculator	☐ Student grade system	☐ Banking console application
☐ Library management system	☐ Employee management system	☐ Spring Boot CRUD API
☐ Employee management REST API	☐ Authentication system	☐ Inventory management
☐ E-commerce backend	☐ Order management microservices	☐ Payment integration service
☐ Kafka event-driven system	☐ Redis caching application	☐ Production-ready SaaS backend

45 What to Prioritise for a Java Backend Job

KEY REVISION NOTES

- Knowing every topic in this guide is not necessary before applying. Build depth in this order, and keep practising DSA alongside from the start.

Priority	Topics	Sections
1	Core Java, OOP, Strings, exceptions	1-10
2	Collections, generics, Streams, lambdas	11-15
3	Multithreading and JVM fundamentals	18-19
4	SQL, JDBC, transactions	21-22
5	Spring Boot, REST APIs, Spring MVC	25-29
6	JPA/Hibernate, SQL optimization	22, 30
7	Spring Security, JWT, OAuth basics	31
8	JUnit, Mockito, integration testing	33
9	Git, Maven, Docker, CI/CD	23-24, 38
10	Microservices, Kafka, Redis, observability	34-37
11	DSA and system design	39-41

- **If you already know Java and Spring Boot basics**, focus on depth rather than repeating fundamentals. These areas are particularly valuable to strengthen:

☐ Collections and concurrency	☐ JVM troubleshooting	☐ SQL performance
☐ Spring transaction behaviour	☐ JPA query optimization	☐ Security
☐ Testing	☐ Microservices	☐ System design

THE JOB-READY BENCHMARK

Can you independently design, implement, test, debug, deploy and explain a production-quality Spring Boot application?

Notes reflect the Java and Spring ecosystem as of mid-2026 (Java 21/25 LTS, Spring Boot 3+). Check the official documentation for version-specific details.